

# 软考系统架构设计师

论文模板全集（基于 H.I.S. BASE HOME 项目）

含使用说明、项目素材库与 9 大主题模板

阅读顺序：先读《使用说明》与《项目素材库》，再按主题查阅

共 11 篇

# 软考「系统架构设计师」论文模板集·使用说明

基于 **H.I.S. BASE HOME 项目**（日本旅游业务系统）真实架构定制。目标：把你自己做过的项目，改造成能套多种题目的论文素材。

## 一、目录结构

| 文件                | 主题                           | 优先级    |
|-------------------|------------------------------|--------|
| 00-项目素材库.md       | 项目真实素材、难点库、通用段落              | ☆☆☆ 必读 |
| 01-微服务架构设计.md     | 微服务拆分、服务治理                   | ☆☆☆    |
| 02-云原生与FaaS架构.md  | AWS Lambda、Serverless、容器化    | ☆☆☆    |
| 03-数据架构与混合存储.md   | MySQL + DynamoDB + S3、读写分离   | ☆☆☆    |
| 04-高并发与高可用架构.md   | 缓存、集群、削峰、容错                  | ☆☆☆    |
| 05-系统安全架构.md      | Spring Security、OAuth2、多语言契约 | ☆☆     |
| 06-系统集成与gRPC.md   | proto 三语言契约、异构集成             | ☆☆☆    |
| 07-架构评估与质量属性.md   | ATAM、质量属性场景、权衡点              | ☆☆☆    |
| 08-批处理架构设计.md     | Spring Batch、幂等、重试           | ☆☆     |
| 09-架构演化与遗留系统改造.md | 架构风格、渐进演进、重构                 | ☆☆     |

只求通过：主攻 **01 / 02 / 03 / 04 / 07**，覆盖近五年约 70% 题面。求高分：再补 **06 / 09**，这两类题目竞争小、容易出彩。

### 与 完整论文/ 的区别（重要）

| 目录         | 性质                                               | 用途           |
|------------|--------------------------------------------------|--------------|
| 论文模板/（本目录） | <b>写作指导</b> ：真题问法、摘要模板、五段框架、填充点、踩坑提醒             | 学「怎么写」，自己动手写 |
| 论文模板/完整论文/ | <b>成品范文</b> ：9 篇可直接背诵的完整论文（试题 + 摘要 + 正文约 2800 字） | 背「写好的」，考前突击  |

**完整论文/** 下每篇均按软考「合格论文」格式撰写，结构为：标题 → 数字提醒 → 试题（含三问）→ 摘要 → 正文（五个小标题）→ 写作提示（含待替换数字清单）。

⚠ 范文中的量化数字**全部是示例值**，源码中并不存在，背诵前必须替换为你项目的真实数据。每篇末尾的「待替换数字清单」已逐项列出需要核对的位置。

三种文件形式（内容完全相同，按场景选用）

| 位置           | 形式           | 适合场景                    |
|--------------|--------------|-------------------------|
| 论文模板/（根目录）   | Markdown 源文件 | 修改内容、被 AI 工具读取          |
| 论文模板/PDF/    | 12 个 PDF     | 打印、精读（含 1 个 64 页全本，带封面） |
| 论文模板/HTML/   | 网页文件         | 浏览器打开，搜索关键词更方便          |
| 论文模板/_tools/ | 转换脚本         | 改完 md 后一键重新生成           |

**推荐用法：**平时用 PDF 或 HTML 阅读；需要改内容时改 `.md`，然后重新生成——**不要直接编辑 PDF。**

重新生成方法（修改 md 之后）

```
cd "E:\BaiduNetdiskDownload\软考高级架构师知识点\13.论文写作\论文模板\_tools"

python md2pdf.py          # 全量重新生成（PDF + HTML + 全本）
python md2pdf.py 01       # 只重新生成文件名含 "01" 的
python check_pdfs.py      # 检查 PDF 页数与字体是否正常
python fix_md_lists.py --dry # 检查 md 列表格式（改了内容后建议跑一次）
```

依赖：Python 3 + `markdown` 库 + Microsoft Edge（已确认本机全部具备）。

二、使用方法（三步）

第一步：读素材库，定「项目口径」

打开 `00-项目素材库.md`，重点看：

- 1. 第一节「项目概况」—— 论文第一段直接改写
- 2. 第四节「难点素材库」（10 个难点）—— 这是论文的弹药
- 3. 第七节「必须补充的数字清单」—— ◆ 标记处全部填上你的真实数据

**⚠ 最关键的一步：**先把数字清单填完再写论文。没有数字的论文，阅卷老师 30 秒就能看出是模板套的。

第二步：按题目选模板

每个主题模板的结构固定为六节：

|          |                  |
|----------|------------------|
| 一、真题常见问法 | ← 对照你的考卷，判断属于哪一种 |
| 二、摘要模板   | ← 直接改写，替换【】里的内容  |
| 三、正文五段框架 | ← 每段都有字数分配和写作要点  |
| 四、本项目填充点 | ← 真实模块/类名/配置，直接抄 |
| 五、可用难点素材 | ← 从素材库挑 2~3 个填进去 |
| 六、踩坑提醒   | ← 这一类题目的特有陷阱     |

第三步：换壳不换骨

考场策略：底稿结构不动，按题目要求置换论述重心。

| 题目要求           | 置换动作                         |
|----------------|------------------------------|
| 「论述你如何进行微服务拆分」 | 重心放「四、填充点」的模块划分理由            |
| 「论述微服务架构的服务治理」 | 重心放「五、难点素材」的契约一致性 + 版本治理     |
| 「论述微服务架构的挑战」   | 重心放「架构演化」——v1/v2 SDK 并存、渐进演进 |

三、备考节奏建议

| 阶段     | 时长  | 任务                                |
|--------|-----|-----------------------------------|
| ① 素材准备 | 2 天 | 填完 00 的数字清单；把 10 个难点用自己的话复述一遍     |
| ② 精写底稿 | 3 天 | 选 01-微服务 写一篇完整的 2800 字，手写（考场是手写！） |
| ③ 横向铺开 | 4 天 | 用同一套素材，改写出 03 04 07 三篇            |
| ④ 限时训练 | 3 天 | 每天 1 篇，严格 120 分钟，控制在 2500~3000 字  |
| ⑤ 补漏   | 2 天 | 挑 02 06 各写一篇，确保 4 选 1 时手里有牌可打     |

手写训练不可省。很多人机打 3000 字只要 40 分钟，手写却要 100 分钟，考场上直接写不完。

四、阅卷评分要点（据历年考情）

| 评分维度   | 权重 | 关键动作                        |
|--------|----|-----------------------------|
| 切题     | 高  | 摘要和正文小标题必须呼应题目要求            |
| 项目真实性  | 高  | 具体业务域、具体技术栈、具体数字            |
| 架构决策能力 | 高  | 写出「为什么选 A 不选 B」的权衡          |
| 结构完整   | 中  | 摘要 + 背景 + 设计 + 实现 + 难点 + 总结 |
| 字数     | 中  | 2500~3000 字                 |

| 评分维度 | 权重 | 关键动作           |
|------|----|----------------|
| 卷面   | 中  | 分段清晰，小标题明确，不涂改 |

## 五、三条写作心法

- 1. **写决策，不写名词** ❌「系统采用了微服务架构，使用了 Docker 和 Kubernetes」✅「考虑到预约、消息、批处理的伸缩特征差异巨大（预约是突发流量、批处理是定时峰值、消息是持续低频），我把它拆分为独立部署单元，使批处理的资源峰值不再影响在线预约接口」
- 2. **写权衡，不写完美** ❌「我们选择了最优方案」✅「该方案牺牲了跨表事务一致性，换来弹性伸缩能力；考虑到旅游预约场景下 跨域事务极少，这个交换是可接受的」
- 3. **写演化，不写一步到位** ❌「系统一开始就是现在这个架构」✅「初期采用单体部署，随着【业务变化】，暴露出【问题】，我们在【阶段】进行了【拆分/重构】，其中【难点】通过【手段】解决」

## 六、关于真实性的提醒

本模板集基于 `F:\his-base-home` 源码调研生成，其中的：

- ✅ **模块划分、类名、配置项、版本号** —— 均来自真实文件，可放心使用
- ◆ **性能数字、业务量、成本** —— 项目源码中**不存在**，模板已全部标注，**必须由你填入真实数据，不要照抄**

配置文件中包含真实的凭证类配置项，写作时**只能写配置项名称与作用**，不得出现任何 Client Secret、Access Key、密码、Issuer 的实际值。

## 七、应急方案（考场上 4 题都不熟时）

- 1. 先排除**题目要求最具体、最窄**的那道（越窄越难写）
- 2. 在剩下 3 题里找**能塞进你素材库难点最多**的那道
- 3. 按该主题模板的「三、正文五段框架」搭骨架（10 分钟）
- 4. 从素材库挑 2 个最贴合的难点填进去
- 5. 宁可写「中等深度 + 完整结构」，不要写「局部很深 + 结构残缺」

论文是**及格性考试**，45 分即可。结构完整、切题、有数字、有本人角色，基本就能过。不要在考场上追求写出完美范文。

祝顺利通过。

## BASE HOME 项目素材库（论文写作的事实来源）

本文件是所有主题模板的底座。其余 01 ~ 09 号模板都从这里取素材。写作顺序建议：先读本文件 → 选定

项目口径 → 再打开对应主题模板填充。

⚠️ 先读这一节：素材可信度说明

论文最忌讳编造。本素材库严格区分两类内容：

| 标记     | 含义                | 处理方式                 |
|--------|-------------------|----------------------|
| ✅ 真实   | 来自项目源码/配置，可直接写进论文 | 放心使用                 |
| ◆ 待你补充 | 项目里没有的数据（尤其是性能指标） | 必须由你填入真实或合理数字，禁止直接照抄 |

**重点提醒：**项目源码中**不存在** QPS、响应时间、并发用户数、数据量等性能数据。论文要求「具体数字」，这部分**只能由你结合真实上线情况填写**。本文件在需要数字的地方一律标注 ◆，不要编造后当成事实。

**脱敏要求：**论文中只能写「配置项名称 + 作用」，不得出现任何真实的 OAuth2 Client Secret、AWS Access Key、Cognito Issuer、数据库密码等值。

一、项目概况（论文第一段直接用）

项目名称：BASE HOME（ベースホーム）

所属企业：H.I.S.（エイチ・アイ・エス）——日本大型旅行社集团

**业务定位：**面向旅行者（旅行者）与营业本部（営業本部）的**旅游业务流程支撑平台**，覆盖预约管理（予約管理）、客户消息（顧客メッセージ）、通知发送（通知配信）、文件管理（ファイル管理）与主数据联动。

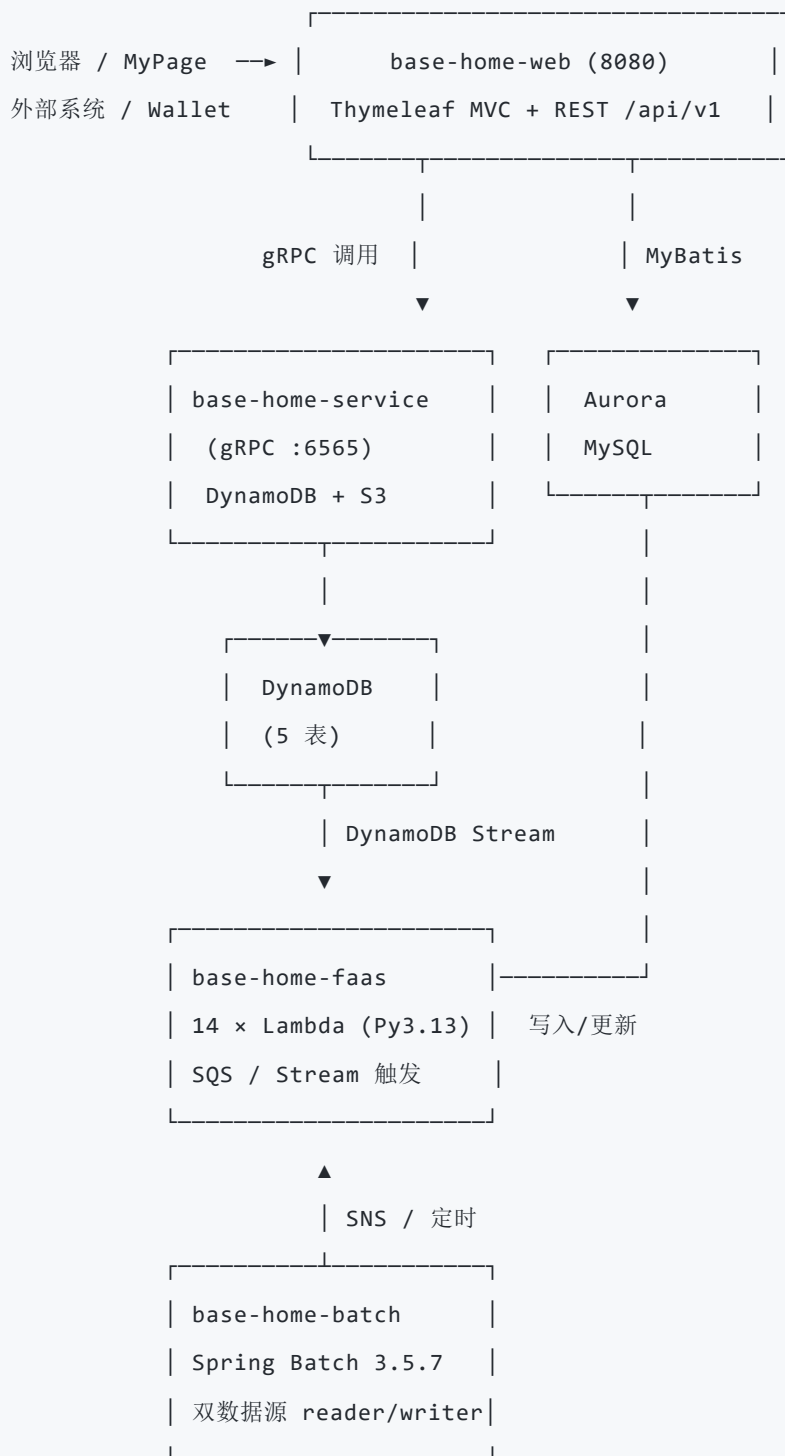
核心业务域（真实 Controller/Service 归类）

| 业务域     | 日文原文    | 承载模块                                      |
|---------|---------|-------------------------------------------|
| 预约管理    | 予約管理    | web（预约搜索/管理/全局搜索）+ service（Travel 系列 RPC） |
| 客户消息    | 顧客メッセージ | web（20+ 消息控制器）+ service（Message RPC 8 个）  |
| 员工消息    | 社員メッセージ | web                                       |
| 通知发送    | 通知配信    | batch（邮件/SNS）+ faas（Wallet 通知、MyPage 通知）  |
| 文件管理    | ファイル管理  | web（S3 下载）+ service（上传/列表 RPC）            |
| 亲权同意书   | 親権同意書   | service（Parental 系列）+ faas                |
| 主数据管理   | マスタ管理   | batch（MDM 联动）+ common（26 张 M 表）           |
| ToDo 管理 | ToDo 管理 | batch + faas + web                        |
| 成本文件    | コストファイル | batch（ACT/AIR/HTL 三类成本文件生成）+ web（下载）      |

项目规模 (  真实)

| 指标               | 数量                                                                          |
|------------------|-----------------------------------------------------------------------------|
| Gradle 模块数       | 8 个 (web / service / batch / common / spec / faas / versions / environment) |
| 独立 Git 仓库        | 各 Java 模块独立仓库, 无顶层聚合                                                        |
| gRPC RPC 方法      | 17 个 ( BaseHomeService ), 服务端 18 个实现类                                       |
| protobuf 文件      | 约 90 个 (enums 13 + request 17 + response 17 + type 约 42)                    |
| MyBatis 实体       | 79 个 (26 张 M* 主数据表 + 53 张 T* 业务表)                                           |
| MyBatis Mapper   | 54 个                                                                        |
| AWS Lambda 函数    | 14 个 (Python 3.13)                                                          |
| DynamoDB 表       | 5 张 (travel / document / parental / sequence / message_thread)              |
| Spring Batch Job | 10 个 (含 1 个测试用 NoopJob)                                                     |

## 二、模块清单（架构图的骨架，✅ 真实）



基座: base-home-common (实体/常量/消息/CSV)

base-home-spec (proto → Java/Go/PHP + OpenAPI 文档)

base-home-versions (Gradle 统一版本管理插件)

各模块要点（写作时按主题取用）

| 模块                 | 技术要点                                                                                                        | 可支撑的论文主题          |
|--------------------|-------------------------------------------------------------------------------------------------------------|-------------------|
| base-home-web      | Spring Boot MVC + Thymeleaf、Spring Security + OAuth2/OIDC、Redis Cluster Session、MyBatis、SpringDoc /api-docs | 安全架构、高可用、系统集成     |
| base-home-service  | gRPC 服务端（@GRpcService）、DynamoDB Enhanced Client、S3、STS AssumeRole、重试                                        | 微服务、数据架构、集成、高可用   |
| base-home-batch    | Spring Batch、双数据源读写分离、SNS、MDM 联动、Prometheus PushGateway、幂等控制                                                | 批处理、数据架构、高可用      |
| base-home-common   | 统一消息体系（EABH00001 类消息 ID）、SuperCSV、79 实体、自动生成的双语 properties                                                  | 数据架构、复用           |
| base-home-spec     | 90 个 proto、一份契约生成 Java/Go/PHP 三语言、OpenAPI + MkDocs                                                          | 系统集成、架构复用、微服务     |
| base-home-faas     | 14 个 Lambda、Powertools、SQS / DynamoDB Stream 触发、SSM Parameter Store、pytest + moto                           | 云原生、FaaS、数据架构、高可用 |
| base-home-versions | Gradle 插件统一版本 + resolutionStrategy force                                                                    | 架构可维护性、构建治理       |

三、技术栈版本（ 真实，可直接引用）

| 组件                         | 版本                                                              |
|----------------------------|-----------------------------------------------------------------|
| Java                       | 21                                                              |
| Spring Boot                | 3.5.7                                                           |
| Spring Batch               | 3.5.7                                                           |
| Gradle                     | 8.10 (wrapper) / 8.14.3 (CI)                                    |
| MyBatis                    | 3.0.5 (强制覆盖 Spring Boot 托管版本)                                   |
| gRPC (io.grpc)             | 1.70.0                                                          |
| protobuf                   | [3,4) 区间，实际 3.25.x                                              |
| proto-google-common-protos | 2.63.2 (强制降级，见难点素材 ②)                                           |
| Redis 客户端                  | Jedis (Redis Cluster 6 节点)                                      |
| AWS SDK                    | v1 + v2 并存 (见难点素材 ③)                                            |
| Python                     | 3.13                                                            |
| Go                         | 1.20                                                            |
| PHP                        | grpc ^1.42.0 / protobuf ^3.19    ^4.26                          |
| 测试                         | TestContainers + LocalStack + H2 + embedded-redis + pytest/moto |

## 四、难点素材库（论文「难点与解决」段的核心弹药）

每个难点都已标注**可套用的主题**。考场上优先选与你题目最贴合的 2~3 个。

### ① 一份 proto 契约驱动三语言客户端

- **问题**：系统需同时为 Java（内部服务）、Go（周边平台）、PHP（既有遗留系统）提供 API。若各自维护接口定义，字段口径必然漂移，联调成本高，回归风险大。
- **方案**：在 `base-home-spec` 中以 protobuf 作为**唯一契约源**，通过 `build.sh` 编排 `build-java → build-go → build-php → generate-all`，分别用 `protoc-gen-grpc-java`、`protoc --go_out/--go-grpc_out/--grpc-gateway_out`、`--php_out + grpc_php_plugin` 生成三语言产物并独立发布（Maven / Go module / Composer）。同时用 `--openapi2_out` 产出 `base_home_api.swagger.json`，经 MkDocs 发布为在线文档。
- **可量化效果**：一份契约、**17 个 RPC** 三端一致；接口变更由构建流水线统一验证，消除了人工同步定义；`base-home-spec` 产出的 90 个 proto 文件全部纳入版本管理。◆ 建议补充：改造前后接口联调耗时对比、字段不一致缺陷数下降比例。
- **可套用主题**：系统集成、微服务、架构复用、架构演化

### ② protobuf 依赖版本冲突导致服务启动失败

- **问题**：`proto-google-common-protos` 升到 `2.64.0-rc1` 后，gRPC 服务启动报错，且因版本区间声明为 `[3,4)`、`grpcVersion = latest.release`，故障在构建期不显现，只在运行时暴露，属于典型的**依赖漂移**引发的架构风险。
- **方案**：在 `base-home-versions` 的统一版本源中，通过 `resolutionStrategy { force ... }` 将 `proto-google-common-protos` 锁定到 **2.63.2**，并同步锁定 MyBatis 3.0.5 等关键组件，把「隐式版本」变成「显式契约」。
- **可量化效果**：启动失败问题消除；构建可复现（同一 commit 任意机器产物一致）。◆ 建议补充：锁定前每月因依赖漂移导致的构建/启动故障次数。
- **可套用主题**：微服务治理、架构可维护性、架构演化、CI/CD（本难点**非常适合**“架构可维护性/演化”类题目，因为它体现“架构决策的长期成本”）

### ③ AWS SDK v1 与 v2 长期并存

- **问题**：项目历史遗留使用 AWS SDK **v1**（S3 / STS），而 DynamoDB 增强客户端（`DynamoDbEnhancedClient`）只在 **v2** 提供，两套 SDK 的凭证链、异常模型、配置方式完全不同，直接替换会波及大量既有代码。
- **方案**：**接受双版本并存的过渡态**，按能力边界划分：新增能力（DynamoDB 增强访问、STS AssumeRole）一律走 v2，既有能力暂留 v1；在 `DynamoDbConfig` / `AwsConfig` 中集中封装客户端构建，统一注入 STS AssumeRole 临时凭证 + `DefaultCredentialsProvider` 兜底。
- **可量化效果**：新功能不再受阻于 SDK 版本；凭证获取统一收口到配置类，一账户一集群。◆ 建议补充：统一凭证后的凭证失效类故障下降情况。
- **可套用主题**：架构演化、遗留系统改造、系统集成（这是“**不追求一次性重写、渐进式演进**”的绝佳论据）

#### ④ MySQL 读写分离双数据源

- **问题：**批处理既要从生产库大量读取（成本文件、ToDo、预约导入），又要回写结果，读写互相争抢资源，长事务影响在线业务。
- **方案：**在 `base-home-batch` 中配置**双数据源**：`ReaderDataSourceConfig`（`spring.datasource.reader`）与 `WriterDataSourceConfig`（`spring.datasource.writer`，标 `@Primary`），通过包路径隔离 Mapper（`repository/reader/` 与 `repository/writer/`），事务隔离级别设为 `TRANSACTION_READ_COMMITTED`，`defaultFetchSize=200` 控制批量游标。
- **可量化效果：**读写物理隔离，批量作业不再挤占在线库连接池。◆ 建议补充：作业耗时、在线接口 P99 响应时间的改善数据。
- **可套用主题：**数据架构、批处理、高可用、性能优化

#### ⑤ DynamoDB 单表多实体 + 混合存储的数据架构

- **问题：**旅游业务数据形态差异极大——旅行主数据（Travel）字段频繁演进、文档（Document）为二进制引用、消息线程（MessageThread）读写频繁、序列号（Sequence）要求强一致自增。若全部塞进关系库，扩展性与弹性不足；若全部 NoSQL，又丢失事务与关联查询能力。
- **方案：混合存储（Polyglot Persistence）：**
- 关系型（Aurora MySQL）承载交易强一致与复杂查询（53 张 `T*` 业务表 + 26 张 `M*` 主数据表）
- DynamoDB 承载高弹性、schema-less 的 5 张表，采用 **分区键 + 排序键** 设计（如 `travel` 表以 `travel_no` 为分区键、`data_type` 为排序键、`json_data` 存半结构化 JSON），实现**单表多实体**，避免表数量膨胀
- S3 承载文件实体（`ap-northeast-1` 区域）
- `Sequence` 表用 `seq_key/seq_no` 提供分布式序列号
- **可量化效果：**`scan-limit=10000` 控制扫描边界；表名带环境前缀，实现「1 账户 1 集群」隔离。◆ 建议补充：单表承载的实体种类数、查询延迟数据。
- **可套用主题：**数据架构、大数据、微服务（每个主题都会问“数据怎么存”）

#### ⑥ Serverless 数据同步链路（最终一致性）

- **问题：**DynamoDB（服务端写入）与 MySQL（报表/既有系统查询）之间需要同步，但同步逻辑若放在 Java 服务里，会拉长主链路响应时间，且扩容成本高。
- **方案：事件驱动的 Serverless 同步：**DynamoDB Stream 与 SQS 作为事件源触发 14 个 Lambda 函数，按业务域拆分（`import_travel_information`、`import_document_information`、`import_message_thread`、`import_booking_information`、`import_mail_information`、`import_parental_consent`、`import_notification`、`import_wallet_status` 等），每个函数独立 Dockerfile、独立 GitHub Actions workflow、独立 SAM 部署，配置从 SSM Parameter Store 读取，用 `aws_lambda_powertools` 做日志/链路追踪/批量部分失败处理。主链路只写 DynamoDB 即返回，同步异步化，达成**最终一致性**。
- **可量化效果：**14 个函数按域独立扩缩容；单个函数故障不影响其他业务域。◆ 建议补充：同步延迟（秒级）、峰值消息处理量、按需计费节省的成本比例。
- **可套用主题：**云原生、FaaS、数据架构、高可用（**近两年最热，强烈建议准备**）

## ⑦ 批量消息推送的幂等与重试

- **问题：**客户消息（顧客メッセージ）常需**批量推送**（`BulkPutCustomerMessage`），邮件发送（`MailSendJob`、`AutomaticEmailCreateJob`）存在网络抖动、第三方邮件网关限流，重试若不加控制会产生**重复发送**，对客户是严重体验事故。
- **方案：**
- 幂等控制：`app.batch.idempotency-duration-minutes=10` 时间窗去重
- 专用重试服务：`MailSendRetryService` 独立承担重试，与业务解耦
- 事务边界：`service/transaction/*TransactionalService` 拆分事务粒度
- 可观测：Prometheus PushGateway（`shutdown-operation: post`）上报作业指标
- **可量化效果：**重复发送被幂等窗口拦截；作业失败可观测、可重跑。◆ 建议补充：重试成功率、重复发送率降到多少。
- **可套用主题：**批处理、高可用、数据一致性

## ⑧ 跨国多时区时间一致性

- **问题：**系统服务日本本部（源库为 **JST**）、部署环境与开发终端时区各异（容器为 **UTC**、开发机为 **UTC+8**）。预约时间、截止时间、邮件发送时刻等一旦跨时区比较出错，会出现「提前发送」或「漏发」——对旅游业务是不可接受的。
- **方案：**确立**时间口径单一化原则**：存储统一 **UTC**，展示按业务方时区（**JST**）转换，转换动作集中到 `FormatUtils` 等工具类，禁止在业务代码中散落 `LocalDateTime.now()`；批处理的 `executionBaseDate`、`acquisitionRange` 作为 `JobParameter` 显式传入，不依赖系统时钟。
- **可量化效果：**时间比较错误集中收口；批处理基准日期可重放（同一参数结果可复现）。◆ 建议补充：改造前时区相关缺陷单数量。
- **可套用主题：**数据架构、批处理、国际化系统架构（**特色素材，别人写不出来，容易出彩**）

## ⑨ 分布式会话与在线扩容

- **问题：**Web 层多实例部署后，Session 若存本地内存，用户会被反复踢下线。
- **方案：**引入 **Spring Session Data Redis**，接 **Redis Cluster（6 节点）**，客户端 Jedis；`spring.session.timeout=3600`。安全侧配合 `spring-session` 与 Spring Security 的会话并发控制。
- **可量化效果：**Web 实例无状态化，可水平扩容；实例重启不掉线。◆ 建议补充：扩容后支撑的并发用户数、会话序列化后的请求延迟。
- **可套用主题：**高可用、高性能、微服务

## ⑩ 主数据统一（MDM）与外部系统联动

- **问题：**H.I.S. 集团存在 MDM（Master Data Management）主数据平台，本国、城市、航空公司、货币等主数据需与 BASE HOME 保持一致，人工维护必然不一致。
- **方案：**`ImportMdmJob` 定时从 MDM API 拉取主数据落库，经 `api/mdm/` 封装认证与查询参数；本地保留 26 张 `M*` 主数据表作为只读副本，业务查询不直连外部。
- **可量化效果：**主数据一致性由作业保障，业务侧零改造。◆ 建议补充：主数据同步频率、覆盖的主数据表数量。

- **可套用主题：**系统集成、数据架构、企业应用集成（EAI）

## 五、论文通用段落（背下来，考场上省 20 分钟）

### 5.1 摘要万能结构（约 300 字）

本文论述了本人参与的【项目名称】的【主题】设计与实现。  
该系统是【一句话业务定位】，服务于【用户群体】，  
采用【核心技术栈】构建，本人担任【角色，务必写“架构师/技术负责人”】，  
负责【3 项具体职责】。  
针对【核心难题 1】、【核心难题 2】两大挑战，  
本文从【架构视角 1】、【架构视角 2】、【架构视角 3】三个维度进行设计，  
通过【关键手段】实现了【量化成果 1】和【量化成果 2】。  
系统上线后【运行效果】，验证了方案的有效性。

**摘要三要素：**项目背景 + 本人角色 + 主要工作与成效。结尾必须落到「成效」，阅卷老师主要看这一段判断你有没有真做过。

### 5.2 项目背景段模板（约 400 字）

【项目名称】是【公司名】面向【用户群体】建设的【系统定位】，  
覆盖【业务域 1】、【业务域 2】、【业务域 3】等核心业务，  
日均处理【◆ 填真实量级】。  
系统自【◆ 时间】启动建设，本人作为架构师参与了整体架构设计，  
主要负责【你的真实职责】。  
项目采用【Java 21 + Spring Boot 3.5.7】技术栈，  
按【多模块 Gradle 结构】划分为 Web、Service、Batch、Common、Spec、FaaS 等模块，  
部署于 AWS【区域】。

### 5.3 结尾段模板（约 200 字）

目前该系统已稳定运行【◆ 时长】，支撑【◆ 业务量】，  
【核心指标】达到【◆ 数值】。  
回顾整个建设过程，仍有可改进之处：  
一是【不足 1】，二是【不足 2】。  
后续计划通过【改进方向】进一步优化。  
通过本项目，本人深刻体会到  
【一句话架构感悟，如：架构设计的本质是在质量属性之间做权衡】。

**结尾必须写「不足」。**不写不足的论文常被认为缺乏架构思维。不足要写得「真诚但无害」，例如「跨语言契约的破坏性变更尚未实现自动化兼容性检测」，而不是「系统经常宕机」。

六、论文写作红线（阅卷扣分点）

| 红线        | 说明                                   |
|-----------|--------------------------------------|
| ✗ 字数不足    | 少于 2000 字基本判不及格，必须练到稳定 2500 ~ 3000 字 |
| ✗ 通篇讲原理   | 架构师论文考「你的决策」，不是考「什么是微服务」             |
| ✗ 没有量化    | 难点必须有数字，没有数字的"难点"是伪难点                |
| ✗ 不写本人角色  | 「我们团队」是减分项，要写「我主导了...我选择了...」        |
| ✗ 不写权衡    | 只写「我选了 A」不写「为什么不是 B」，是初级工程师视角        |
| ✗ 摘要与正文脱节 | 摘要里提到的三个维度，正文必须有对应小标题                |
| ✗ 技术堆砌无逻辑 | 列出 10 个技术名词却不解释它们如何协同，是典型外行写法        |
| ✓ 加分项     | 写架构演化过程（从 A 演进到 B，为什么）、写失败与修正        |

七、◆ 必须由你补充的「数字清单」

论文想拿高分，下面这些数据必须补全。请对照真实情况填写，**不要编造**：

| 类别   | 需要的数据                       | 建议来源               |
|------|-----------------------------|--------------------|
| 业务规模 | 日均预约量、注册旅行者数、消息日推量          | 运营报表               |
| 用户规模 | 营业本部使用人数、门店数、并发用户数          | 业务方                |
| 性能   | 接口 P99 响应时间、批处理作业耗时、同步延迟    | APM / Prometheus   |
| 可用性  | SLA 目标、年度可用率、故障恢复时间（MTTR）   | 运维记录               |
| 容量   | 数据库数据量、DynamoDB 表条目数、S3 存储量 | 云监控                |
| 成本   | 上云前后成本对比、Serverless 节省比例    | 财务 / Cost Explorer |
| 质量   | 缺陷率下降、重复发送率、数据不一致单数         | 质量报告               |
| 时间   | 项目启动时间、上线时间、稳定运行时长          | 项目计划               |

**写作技巧：**数据可以不精确，但必须**自洽**。例如说「日均 3 万笔预约」，后文就不能出现「日处理 300 笔」。建议先在纸上把口径统一，再动笔写正文。

# 论文模板 01 · 微服务架构设计

配套素材见 [00-项目素材库.md](#) 。本模板所有  内容均来自 BASE HOME 真实源码。

## 一、真题常见问法

### 变体 A（架构设计型）

请论述你参与的某系统的微服务架构设计，包括服务拆分原则、服务间通信方式、服务治理机制，并说明你在设计过程中遇到的挑战及解决方案。

### 变体 B（拆分决策型）

微服务架构的核心难点之一是服务粒度的划分。请结合你参与的项目，论述你如何确定服务边界，以及拆分过程中如何保证数据一致性。

### 变体 C（挑战型）

请论述微服务架构在实际落地中面临的挑战（如分布式事务、服务依赖、版本管理），并结合项目说明你的应对措施。

**判断：**题目若强调「拆分/边界」→ 重心放服务划分理由；强调「治理/挑战」→ 重心放契约一致性 + 版本治理 + 最终一致性。

## 二、摘要模板（约 300 字）

本文论述了本人参与的 H.I.S. BASE HOME 旅游业务平台的微服务架构设计与实现。

该平台面向日本 H.I.S. 集团的营业本部与旅行者，覆盖预约管理（予約管理）、客户消息（顧客メッセージ）、通知发送（通知配信）与主数据联动等核心业务。

本人担任架构师，负责服务边界划分、服务间通信契约设计与构建治理体系。

针对【业务伸缩特征差异大】、【跨语言调用契约易漂移】、【新旧技术栈长期并存】三大挑战，

本文从【服务拆分】、【契约驱动】、【渐进演进】三个维度进行设计：

按业务特征与伸缩曲线将系统划分为 Web、Service、Batch、FaaS 等独立部署单元，

以 Protocol Buffers 作为唯一契约源驱动 Java、Go、PHP 三语言客户端生成，

并通过统一版本管理插件锁定关键依赖，规避了依赖漂移引发的运行时故障。

系统上线后【 填效果】，验证了方案的可行性。

### 三、正文五段框架

#### 第 1 段·项目背景与本人角色 (约 400 字)

写什么：

- 项目名称、所属企业、业务定位、服务对象
- 项目规模（用素材库第二节的数字：8 个模块、17 个 RPC、79 个实体、14 个 Lambda）
- 本人角色与 3 项具体职责
- 一句话点出「为什么需要微服务」（引出后文）

本项目可直接用：

BASE HOME 是 H.I.S. 面向旅游业务建设的支撑平台，覆盖预约管理、客户消息、通知发送与主数据联动等业务域，采用 Java 21 + Spring Boot 3.5.7 技术栈，按多模块 Gradle 结构划分为 Web、Service、Batch、Common、Spec、FaaS 等模块，部署于 AWS 东京区域（ap-northeast-1）。本人担任架构师，负责服务边界划分、服务间通信契约设计以及构建版本治理。

#### 第 2 段·架构设计 (约 800 字，本文核心)

必须先讲「拆分依据」——这是区分架构师与工程师的关键。

推荐三个拆分维度（本项目真实成立）：

##### 维度一：按伸缩特征拆分

Web 层承载营业本部员工的交互操作，流量呈**工作日昼间高峰**特征；Batch 层承载成本文件生成、邮件发送等定时作业，呈**夜间定时峰值**且瞬时资源占用大；FaaS 层承载数据同步，呈**持续低频**特征。若合并部署，批处理的资源峰值会直接冲击在线预约接口的响应时间。因此我将三者拆分为独立部署单元，使各层可按自身曲线独立扩缩容。**权衡说明**：这牺牲了单体部署的调用简单性，换来了资源隔离与独立伸缩能力。

##### 维度二：按访问协议与调用方拆分

内部服务间采用 **gRPC**（`base-home-service` 暴露 6565 端口，17 个 RPC），而非 REST。原因是服务间调用对性能与强类型契约更敏感，gRPC 基于 HTTP/2 多路复用且由 proto 保证强类型；对外则通过 Web 层的 REST 接口（`/api/v1/**`）暴露，兼顾浏览器友好性与前后端契约稳定性。

##### 维度三：按数据存储特征拆分（本项目最有说服力的一点）

我采用**混合存储 (Polyglot Persistence)**：Aurora MySQL 承载强一致交易与复杂关联查询（79 张业务/主数据表）；DynamoDB 承载 5 张 schema-less 高弹性表（travel / document / parental / sequence / message\_thread）；S3 承载文件实体。服务边界与数据边界对齐，避免多个服务共享同一张表造成的隐性耦合。

画架构图（文字版，考场上画方框箭头即可）：



### 第 3 段 • 关键技术实现 (约 700 字)

#### 要素一：契约驱动的服务通信

系统需同时为 Java、Go、PHP 三种技术栈的调用方提供服务。我在 `base-home-spec` 模块中以 `protobuf` 作为**唯一契约源**，通过构建脚本编排 `build-java → build-go → build-php → generate-all`，分别生成三语言产物并独立发布（Maven / Go module / Composer）。同时用 `--openapiv2_out` 产出 OpenAPI 描述文件，经 MkDocs 发布为在线文档。该设计使「17 个 RPC 的接口口径」在三端天然一致，把接口不一致问题从「运行时联调发现」提前到「构建期暴露」。

#### 要素二：服务端的分发与复用

`base-home-service` 采用单一入口 `BaseHomeService`（`@GRpcService`）承接 17 个 RPC，统一经 `execute(IGrpcService, req, respObserver)` 方法分发到各业务实现类，共 18 个实现类；跨业务域的能力（文档、消息线程、亲权同意书、序列号、旅行信息）下沉到 `service/shared/` 复用，避免同一逻辑在多个 RPC 中重复实现。

#### 要素三：构建期依赖治理

项目通过独立的 `base-home-versions` Gradle 插件统一版本源（`base-home.versions.gradle`），各模块以 `apply plugin: 'base-home.versions'` 接入，并用 `resolutionStrategy { force ... }` 锁定关键组件。

### 第 4 段 • 难点与解决 (约 700 字)

从素材库第四节挑 2 ~ 3 个。微服务主题的最佳组合：

| 优先级 | 难点                       | 为什么适合微服务主题       |
|-----|--------------------------|------------------|
| 首选  | ① 一份 proto 契约驱动三语言       | 直接对应「服务间通信」考点    |
| 次选  | ② protobuf 版本冲突致启动失败     | 直接对应「服务治理」考点     |
| 三选  | ⑥ Serverless 数据同步（最终一致性） | 直接对应「分布式数据一致性」考点 |

难点写法三段式（每个难点 200 字左右）：

问题 → 为什么这是个架构问题（不是编码问题）  
方案 → 你的决策 + 为什么选它（对比其他选项）  
效果 → ◆ 量化数字

示例（难点②）：

我们在升级 protobuf 依赖时，`proto-google-common-protos` 被解析到 2.64.0-rc1，导致 gRPC 服务启动直接失败。更棘手的是，由于版本声明采用了动态区间（`protobufVersion = '[3,4)'`、`grpcVersion = 'latest.release'`），该问题在构建期完全不显现，只在运行时暴露——这属于典型的依赖漂移风险。我的判断是：这不是一次偶发故障，而是「版本契约不明确」这一架构缺陷的必然结果。因此我没有只做局部修复，而是在统一版本源中将该组件强制锁定到 2.63.2，并同步锁定 MyBatis 3.0.5 等关键组件，把隐式版本变成显式契约。改造后构建具备可复现性，同一提交在任何机器上产物一致。

## 第 5 段 · 总结与不足（约 200 字）

目前 BASE HOME 已稳定支撑【◆ 业务量】，【◆ 核心指标】达到【◆ 数值】。  
回顾建设过程，仍存在两点不足：  
一是跨语言契约虽有统一来源，但破坏性变更尚未实现自动化的兼容性检测；  
二是 AWS SDK v1 与 v2 仍处于并存状态，长期维护成本尚未完全消除。  
后续计划通过引入契约兼容性校验工具、逐步收敛 SDK 版本进一步优化。  
通过本项目，我深刻体会到微服务架构的价值不在于“拆”本身，  
而在于让每个服务能够按自身的业务节奏独立演进。

## 四、本项目填充点速查

| 论文要素   | 直接可写的内容                                                                    |
|--------|----------------------------------------------------------------------------|
| 服务划分单元 | Web / Service / Batch / Common / Spec / FaaS                               |
| 通信方式   | 服务间 gRPC (HTTP/2 + 强类型契约)；对外 REST <code>/api/v1/**</code>                  |
| RPC 数量 | 17 个 RPC，18 个服务端实现类                                                        |
| 服务内复用  | <code>service/shared/</code> 5 个共享服务                                       |
| 契约管理   | protobuf 唯一契约源，Java/Go/PHP 三语言 + OpenAPI 文档                                |
| 数据边界   | Aurora MySQL (79 表) + DynamoDB (5 表) + S3                                  |
| 版本治理   | <code>base-home-versions</code> 插件 + <code>resolutionStrategy force</code> |
| 独立部署   | 每个 Lambda 独立 Dockerfile + 独立 GitHub Actions                                |
| 环境隔离   | DynamoDB 表名带环境前缀，「1 账户 1 集群」                                               |
| 本地研发   | TestContainers + LocalStack ( <code>:4566</code> ) 模拟 AWS                  |

| 论文要素 | 直接可写的内容                                                                 |
|------|-------------------------------------------------------------------------|
| 依赖方向 | web → common + spec; service → common + spec; batch → common + mdm-spec |

## 五、可用难点素材（按适配度排序）

- 1. ① 一份 proto 契约驱动三语言 ★★★ —— 服务间通信、契约一致性
- 2. ② protobuf 版本冲突致启动失败 ★★★ —— 服务治理、依赖漂移
- 3. ⑥ Serverless 数据同步与最终一致性 ★★★ —— 分布式数据一致性
- 4. ③ AWS SDK v1/v2 并存 ★★ —— 渐进演进（也可放“挑战”类题目）
- 5. ⑨ 分布式会话 Redis Cluster ★★ —— 有状态服务的无状态化

## 六、踩坑提醒

| 坑                | 后果     | 正确做法                                                          |
|------------------|--------|---------------------------------------------------------------|
| 只写「我们用了微服务」      | 判定为背概念 | 必须写 <b>拆分依据</b> ，且依据要来自业务特征                                   |
| 大谈 CAP、BASE 理论   | 空洞     | 用一个具体场景说明最终一致性（如 DynamoDB Stream → Lambda → MySQL）            |
| 把模块当微服务数         | 逻辑矛盾   | 8 个 Gradle 模块 ≠ 8 个微服务，说清「部署单元」与「代码模块」的区别                     |
| 不提代价             | 缺乏架构思维 | 明确写出微服务带来的成本：分布式调试难、跨服务事务缺失、运维复杂度上升                           |
| 忽略数据边界           | 关键失分点  | 服务拆分必须谈数据拆分，本项目「混合存储」是绝佳论据                                    |
| 说的是「服务注册发现」但项目没用 | 编造风险   | 本项目未用服务注册中心（gRPC 直连 endpoint + TLS）， <b>不要编造 Eureka/Nacos</b> |

⚠️ **最后一条最重要**：BASE HOME 的 gRPC 调用是**配置化 endpoint 直连**（`tls://dev-base-home.hisgroup.app:6565`），并非通过注册中心发现。论文中若写「我们使用 Nacos 做服务发现」，一旦被追问细节就会露馅。诚实写「服务地址通过配置中心化下发，配合 TLS 保证传输安全」同样成立且更真实。

配套：[00-项目素材库.md](#) · [06-系统集成与gRPC.md](#)（契约部分可互相复用）

# 论文模板 02 · 云原生与 Serverless（FaaS）架构

配套素材见 [00-项目素材库.md](#)。本主题**近三年出现频率极高**，强烈建议准备。

## 一、真题常见问法

### 变体 A（云原生整体）

请论述你参与的某系统的云原生架构设计，包括容器化、微服务、持续交付、DevOps 等方面的实践，并说明其带来的价值。

### 变体 B（Serverless 专项）

请论述 Serverless（无服务器）架构的适用场景与设计要点，结合你参与的项目说明如何划分函数粒度、如何处理冷启动与状态管理。

### 变体 C（弹性与成本）

请论述你如何利用云平台能力实现系统的弹性伸缩与成本优化。

**判断：**题目提「容器/K8s/CI-CD」→ 走变体 A 路线；提「函数计算/事件驱动/按量付费」→ 走变体 B 路线（本项目 Lambda 素材极强）。

## 二、摘要模板（约 300 字）

本文论述了本人参与的 H.I.S. BASE HOME 旅游业务平台的云原生架构设计与实现。

该平台部署于 AWS 东京区域，承载预约管理、客户消息、通知发送等业务。

本人担任架构师，负责容器化方案设计、Serverless 数据同步链路设计与交付流水线建设。

针对【数据同步与主链路解耦】、【函数粒度划分】、【多云资源治理】三大挑战，

本文从【事件驱动解耦】、【按业务域拆分函数】、【基础设施即代码】三个维度进行设计：

以 DynamoDB Stream 与 SQS 作为事件源，驱动 14 个按业务域划分的 Lambda 函数

完成数据同步，使主链路仅写 DynamoDB 即可返回，达成分钟级最终一致性；

每个函数独立镜像、独立流水线、独立部署，实现故障域隔离。

系统上线后【 填效果】，验证了方案的可行性。

## 三、正文五段框架

### 第 1 段·项目背景与为何选择云原生（约 400 字）

**关键：**不要一上来就说「我们上云了」，要写驱动力。

旅游业务的典型特征是**强季节性**：长假前预约量激增，平峰期骤降。传统固定容量的部署方式在旺季需要按峰值配置资源，淡季则大量闲置，资源利用率长期偏低。这一矛盾是推动我们采用云原生架构的直接动因。

本项目可直接用：

BASE HOME 部署于 AWS ap-northeast-1，采用 Java 21 + Spring Boot 3.5.7，各模块以容器镜像方式交付（基础镜像统一为内部 Java 21 镜像），数据同步链路由 14 个 Python 3.13 Lambda 函数承担，配置统一存放于 SSM Parameter Store，避免密钥硬编码。

## 第 2 段 · 架构设计 (约 800 字)

分三层论述：

### 第一层：容器化与交付

Web (8080)、Service (gRPC 6565)、Batch (无 Web 容器，`spring.main.web-application-type: none`) 各自独立 Dockerfile 与镜像，镜像基于集团统一基础镜像，保证运行时基线一致。本地研发环境通过 TestContainers 拉起 MySQL，通过 LocalStack (`:4566`) 模拟 DynamoDB 与 S3，使「本地能跑通」与「云上能跑通」的技术栈一致，消除了环境差异导致的缺陷。

### 第二层：Serverless 数据同步 (本文核心)

系统的写入主链路落在 DynamoDB，而报表与既有系统仍需从 MySQL 查询。若在 Java 服务中同步写两侧，主链路响应时间会被拉长，且同步逻辑的扩容需求会绑架整个服务的资源配置。因此我把同步链路设计为**事件驱动的 Serverless 架构**：

- **事件源**：DynamoDB Stream (写后事件) + SQS (业务消息队列)
- **计算单元**：14 个 Lambda 函数，按业务域拆分
- **配置源**：SSM Parameter Store
- **可观测**：`aws_lambda_powertools` 提供结构化日志、Tracer 链路追踪与 SQS/Stream 批量处理工具

按触发方式可分为两类：| 触发方式 | 函数 | |---|---| | DynamoDB Stream |

`import_document_information`、`import_message_thread`、`import_parental_consent`、  
`import_travel_information`、`send_travel_information` || SQS | `import_booking_information`、  
`import_mail_information`、`import_notification`、`import_wallet_notification`、  
`import_wallet_status`、`update_travel_status` || 定时 | `send_email_footer`、  
`send_email_template`、`send_todo_information` |

**权衡说明**：该设计以**最终一致性**替代强一致性——主链路返回时，MySQL 侧数据尚未更新。评估后我们认为对报表与通知场景可接受，而预约写入等强一致场景仍走关系库事务，不做异步化。

### 第三层：函数粒度划分原则

我没有按「一张表一个函数」划分，而是按**业务域**划分。理由：同一业务域的实体更新往往需要一起处理（如旅行信息变更需同步申请人与预约数据），按表拆会导致函数间编排复杂、且同一业务逻辑被多次触发。按域拆分使每个函数内部可完成完整的业务处理，同时故障域按业务隔离——某个域的函数异常不会波及其他业务。

第 3 段 • 关键技术实现 (约 700 字)

要素一：事件驱动的解耦

主链路只写 DynamoDB 即返回，同步由 Stream 异步接管，把「写数据库」与「同步数据」两个关注点彻底解耦。◆ 补充：主链路响应时间从 X ms 降到 Y ms。

要素二：批量处理与部分失败

Lambda 处理 SQS/Stream 批次时，单条记录失败不应导致整批重试。我们使用 `aws_lambda_powertools` 的 batch 处理工具，支持**部分批次失败**语义，仅将失败记录投入死信队列，正常记录正常提交。

要素三：配置与密钥管理

所有配置（数据库连接、SNS Topic ARN、外部端点）统一从 SSM Parameter Store 读取（`aws/ssm_client.SsmClient`），代码仓库中不落任何明文凭证。服务端侧同样采用 STS AssumeRole 获取临时凭证 + `DefaultCredentialsProvider` 兜底，避免长期凭证泄露风险。

要素四：独立交付流水线

每个 Lambda 拥有独立的 Dockerfile 与独立的 GitHub Actions 工作流（按 `import-*` / `send-*` / `update-travel-status` 命名），支持单函数灰度与回滚，部署动作通过 SAM 完成。这带来一个重要收益：**单个函数的变更不需要整体回归**。

第 4 段 • 难点与解决 (约 700 字)

云原生主题最佳组合：

| 优先级 | 难点                      | 适配理由                  |
|-----|-------------------------|-----------------------|
| 首选  | ⑥ Serverless 数据同步链路     | 主题核心                  |
| 次选  | ⑦ 批量推送的幂等与重试            | 体现 Serverless 下的可靠性设计 |
| 三选  | ⑨ 分布式会话 (Redis Cluster) | 体现「无状态化」这一云原生核心原则     |
| 加分  | ⑧ 跨国多时区一致性              | 跨国部署的特色问题，容易出彩        |

难点示例 (⑥)：

挑战在于**同步延迟与顺序性**。DynamoDB Stream 保证同一分区的记录有序，但跨分区不保证。若两个业务域的数据存在先后依赖，并行处理可能产生顺序错乱。我们的应对是：把存在依赖关系的实体收敛到同一分区键下（如旅行信息统一以 `travel_no` 为分区键），借由 Stream 的分区内有序性保证处理顺序；对确实跨域的场景，采用**幂等 + 版本号**的设计，使乱序到达的更新可以通过比较版本号判定是否过期，避免旧数据覆盖新数据。

第 5 段 • 总结与不足 (约 200 字)

目前 14 个函数已稳定支撑数据同步，主链路响应时间显著下降，且各函数按需计费，平峰期成本大幅降低。

不足之处有两点：

一是冷启动在低频函数上仍有可感知延迟，尚未引入预置并发；

二是函数间缺少统一的编排视图，跨域数据流向的全局监控还不够直观。

后续计划通过预置并发与链路追踪可视化进一步完善。

通过本项目，我认识到 `Serverless` 并非“银弹”——它适合事件驱动、无状态、突发型的负载，而强一致的事务场景仍需依托关系型数据库。

四、本项目填充点速查

| 论文要素          | 直接可写的内容                                                         |
|---------------|-----------------------------------------------------------------|
| 云平台           | AWS ap-northeast-1 (东京)                                         |
| 容器化           | 各模块独立 Dockerfile，统一 Java 21 基础镜像；Service 暴露 6565                |
| 本地云环境         | LocalStack ( :4566 ) 模拟 DynamoDB/S3；TestContainers 模拟 MySQL     |
| Serverless 规模 | 14 个 Lambda 函数，Python 3.13                                      |
| 运行时框架         | <code>aws_lambda_powertools</code> (Logger / Tracer / batch 处理) |
| 事件源           | DynamoDB Stream (5 个函数) + SQS (6 个函数) + 定时 (3 个函数)              |
| 配置管理          | SSM Parameter Store ( <code>SsmClient</code> )                  |
| 凭证管理          | STS AssumeRole 临时凭证 + DefaultCredentialsProvider                |
| 交付方式          | 每函数独立 Dockerfile + 独立 GitHub Actions + SAM 部署                   |
| 批处理容器         | Batch 模块 <code>web-application-type: none</code> ，无端口占用         |
| 监控            | Prometheus PushGateway (Batch) ； Powertools Tracer (FaaS)       |
| 存储服务          | S3 (ap-northeast-1)、DynamoDB (5 表)、Aurora MySQL                 |
| 消息服务          | SNS ( <code>app.services.aws.sns</code> )、SQS                   |
| 单元测试          | pytest + moto (模拟 AWS 服务)，覆盖率纳入 CI                              |

五、可用难点素材 (按适配度排序)

- 1. ⑥ `Serverless` 数据同步与最终一致性 ★★★★★ —— 本主题首选
- 2. ⑦ 批量推送的幂等与重试 ★★★★★ —— 可靠性设计
- 3. ⑨ 分布式会话 Redis Cluster ★★ —— 无状态化

4. ⑧ 跨国多时区一致性 ★★ ——— 跨国部署特色，强烈建议写
5. ③ AWS SDK v1/v2 并存 ★★ ——— 云 SDK 演进

## 六、踩坑提醒

| 坑                 | 后果       | 正确做法                                                            |
|-------------------|----------|-----------------------------------------------------------------|
| 把 Serverless 说成万能 | 缺乏判断力    | 明确写出 <b>不适用场景</b> （强一致事务、长时任务、高频稳定负载）                           |
| 不提冷启动             | 明显漏考点    | 主动提，并说明你的缓解手段（本文用「低频函数可接受延迟」+ 预置并发作为改进方向）                       |
| 编造 K8s 使用经历       | 高风险      | 本项目 <b>未见 K8s 编排证据</b> ，不要写「我们用 K8s 做服务编排」；写 Docker 镜像 + 独立部署即可 |
| 编造服务网格 (Istio)    | 高风险      | 同上，本项目无 Service Mesh                                            |
| 只讲成本不讲复杂度         | 片面       | 写出 Serverless 的代价：本地调试困难、分布式追踪复杂、函数间依赖不直观                       |
| 忽略 IaC            | 漏考点      | 本项目用 SAM 部署、GitHub Actions 流水线，可提「基础设施即代码」                      |
| 数字全靠编             | 阅卷老师能看出来 | 只填你真实知道的；不确定的量级宁可写相对值（如「下降约 40%」）                               |

**⚠ 技术真实性红线：**BASE HOME 使用 Docker + Lambda + SAM + GitHub Actions，**但没有 Kubernetes、没有 Istio、没有服务网格**。若题目明确问 K8s，请评估后改用其他题目，或如实写「容器编排需求较轻，以单容器部署为主，未引入 K8s」——这本身就是一个合理的架构决策。

配套：[01- 微服务架构设计.md](#) · [03- 数据架构与混合存储.md](#)（同步链路的另一端）

## 论文模板 03 · 数据架构与混合存储设计

配套素材见 [00- 项目素材库.md](#)。本主题**近三年明显升温**，且 BASE HOME 素材极佳。

### 一、真题常见问法

变体 A（数据架构整体）

请论述你参与的某系统的数据架构设计，包括数据模型设计、存储选型、数据一致性与数据治理等方面。

变体 B（存储选型）

请结合项目论述你在关系型数据库与 NoSQL 数据库之间的选型依据， 以及如何保证多存储之间的一致性。

变体 C（大数据/高并发数据）

请论述你如何应对系统面临的海量数据存储与高并发访问挑战。

**判断：**变体 B 是 BASE HOME 的**最优命中区**——因为项目真实采用了 MySQL + DynamoDB + S3 三种存储，有真实选型依据可写。

二、摘要模板（约 300 字）

本文论述了本人参与的 H.I.S. BASE HOME 旅游业务平台的数据架构设计与实现。  
该平台承载预约管理、客户消息、通知发送等业务，数据形态差异极大：  
既有强一致的交易数据，也有字段频繁演进的半结构化数据与文件实体。  
本人担任架构师，负责数据模型设计、存储选型与数据同步链路设计。  
针对【单一存储难以兼顾强一致与弹性扩展】、【异构存储间一致性保障】、  
【批量作业与在线业务资源争抢】三大挑战，  
本文从【混合存储选型】、【事件驱动同步】、【读写分离】三个维度进行设计：  
以 Aurora MySQL 承载强一致交易（79 张业务与主数据表），  
以 DynamoDB 承载高弹性半结构化数据（5 张表，单表多实体设计），  
以 S3 承载文件实体，并通过 DynamoDB Stream 驱动 Serverless 链路完成最终一致同步。  
系统上线后【 填效果】，验证了方案的可行性。

三、正文五段框架

第 1 段·项目背景与数据挑战（约 400 字）

核心是把「数据形态差异」讲清楚，这是选型的唯一依据。

旅游业务的数据形态差异极大，我将其归纳为四类：① **强一致交易数据**——预约、申请人、旅行者、费用，要求事务与关联查询；② **频繁演进的半结构化数据**——旅行主信息字段随业务政策频繁增删，若用关系表承载则每次变更都需 DDL；③ **文件实体**——护照、文档、亲权同意书，体积大且需对象存储能力；④ **高频读写的小数据**——消息线程、序列号，读写频繁且要求低延迟。用单一存储承载这四类数据，必然在某一方面做出妥协。

第 2 段·架构设计（约 800 字，本文核心）

第一层：混合存储选型（Polyglot Persistence）

| 数据类型    | 存储选型                       | 选型依据                                                         |
|---------|----------------------------|--------------------------------------------------------------|
| 强一致交易   | Aurora MySQL               | 需要事务、外键、复杂关联查询；79 张表（26 张 <b>M*</b> 主数据 + 53 张 <b>T*</b> 业务） |
| 半结构化/弹性 | DynamoDB 5 张表              | schema-less、按需扩缩、毫秒级延迟，字段演进无需 DDL                            |
| 文件实体    | S3 (ap-northeast-1)        | 对象存储，与数据库解耦，降低库体积与备份成本                                       |
| 分布式序列号  | DynamoDB <b>sequence</b> 表 | 需跨实例唯一且自增，关系库自增无法满足分布式场景                                     |

第二层：DynamoDB 单表多实体设计（本项目的技术亮点）

我采用**分区键 + 排序键 + JSON 载荷**的模式实现单表多实体：以 **travel** 表为例，分区键为 **travel\_no**（旅行编号），排序键为 **data\_type**（数据类型），实体内容存于 **json\_data** 字段。这样一张表可承载同一旅行下的多种数据类型，既避免了表数量随业务类型线性膨胀，又保证同一旅行的所有数据落在同一分区，**查询时可用一次 Query 取回**，无需跨分区扫描。

其余四张表：  
- **document** —— 文档元数据（文件实体在 S3）  
- **parental** —— 亲权同意书（親權同意書）  
- **message\_thread** —— 消息线程  
- **sequence** —— 序列号（**seq\_key** / **seq\_no**），为分布式环境提供唯一编号

**治理措施**：表名统一带环境前缀，实现「1 账户 1 集群」的环境隔离；**scan-limit** 设为 10000，为全表扫描设置硬边界，防止误用 Scan 拖垮表。

第三层：批处理读写分离

批处理既要大量读取（成本文件生成、外部预约导入、ToDo 状态更新），又要回写结果。我为其配置**双数据源**：  
**ReaderDataSourceConfig**（**spring.datasource.reader**）与 **WriterDataSourceConfig**（**spring.datasource.writer**，标 **@Primary**），通过包路径隔离 Mapper（**repository/reader/** 与 **repository/writer/**），使读写请求物理分离，批量作业不再争抢在线业务的连接池资源。同时设置事务隔离级别为 **TRANSACTION\_READ\_COMMITTED**，避免长事务期间大量持有间隙锁影响在线写入；  
**defaultFetchSize = 200** 控制批量游标，防止一次性载入海量结果集导致 OOM。

第 3 段 · 关键实现（约 700 字）

要素一：数据访问层设计

MySQL 侧统一使用 MyBatis（3.0.5，在统一版本源中强制锁定，覆盖 Spring Boot 托管版本以保证行为一致），79 个实体按职责分两个 source set 组织，主数据实体与业务实体分离；54 个 Mapper 按业务域归入 **repository/**。DynamoDB 侧通过 **DynamoDbEnhancedClient** 提供对象映射，由 **DynamoDbRepository** 基类封装通用操作，实体以 **@DynamoDbBean** 注解声明，属性名集中定义在 **consts** 包（**TravelConsts**、**DocumentConsts** 等），避免魔法字符串散落。

要素二：一致性与重试

DynamoDB 侧启用乐观锁重试（@EnableRetry，重试 4 次、间隔 500ms），应对并发写入冲突。跨存储一致性通过事件驱动链路达成最终一致（DynamoDB Stream → Lambda → MySQL）。

要素三：CSV 与文件下载

成本文件（ACT 原价 / AIR 机票 / HTL 酒店）使用 SuperCSV 注解库 将实体直接映射为 CSV，配合 CsvDownloader 接口与 FileUploadService 落 S3，避免大文件在内存中拼装。导出列定义通过注解集中在实体上，新增列不会波及导出逻辑。

要素四：主数据治理

集团 MDM 主数据由 ImportMdmJob 定时同步，本地保留 26 张 M\* 表作为只读副本，业务查询不直连外部系统——既保证主数据一致性，又隔离了外部系统故障对业务的影响。

第 4 段 · 难点与解决 (约 700 字)

数据架构主题最佳组合：

| 优先级 | 难点                      | 适配理由         |
|-----|-------------------------|--------------|
| 首选  | ⑤ DynamoDB 单表多实体 + 混合存储 | 主题核心         |
| 次选  | ④ MySQL 读写分离双数据源        | 直接对应「数据访问优化」 |
| 三选  | ⑧ 跨国多时区时间一致性            | 数据一致性维度，特色鲜明 |
| 加分  | ⑩ MDM 主数据统一             | 数据治理维度       |

难点示例 (⑧ 多时区，非常出彩)：

系统服务日本本部（源库运行于 JST），而容器环境为 UTC、开发终端为 UTC+8。三套时区并存时，预约截止时间、邮件发送时刻等跨时区比较极易出错，后果是「提前发送」或「漏发」——对旅游业务属于严重事故。我的处理原则是**时间口径单一化**：存储层统一使用 UTC；展示层按业务方时区（JST）转换；转换动作集中到 FormatUtils 等工具类，禁止在业务代码中散落直接取当前时间的调用。批处理侧更彻底——executionBaseDate、acquisitionRange 等作为 JobParameter 显式传入，不依赖系统时钟，使同一参数的重跑结果完全可复现。◆ 补充：改造前时区相关缺陷数量与改造后的下降情况。

第 5 段 • 总结与不足 (约 200 字)

目前该数据架构已稳定支撑【◆ 数据量】，DynamoDB 侧查询延迟稳定在【◆ 数值】。

不足之处：

一是混合存储引入了运维复杂度，多套存储的备份与容量规划需要分别管理；

二是跨存储的最终一致链路缺少统一的延迟监控，异常时的排查路径较长。

后续计划引入统一的数据链路追踪与一致性校验作业。

通过本项目我认识到，数据架构设计没有"最好的存储"，只有"与数据访问特征最匹配的存储"。

四、本项目填充点速查

| 论文要素   | 直接可写的内容                                                                 |
|--------|-------------------------------------------------------------------------|
| 关系库    | Aurora MySQL, 79 张表 (26 M* 主数据 + 53 T* 业务) , 54 个 Mapper                |
| NoSQL  | DynamoDB 5 张表: travel / document / parental / sequence / message_thread |
| 单表多实体  | travel 表: 分区键 travel_no + 排序键 data_type + json_data                     |
| 序号     | sequence 表 ( seq_key / seq_no ) , 分布式唯一编号                               |
| 对象存储   | S3, 区域 ap-northeast-1                                                   |
| ORM    | MyBatis 3.0.5 (强制锁定) ; DynamoDB Enhanced Client                         |
| 读写分离   | reader/writer 双数据源, 包路径隔离 Mapper                                        |
| 事务隔离   | TRANSACTION_READ_COMMITTED                                              |
| 批量游标   | mybatis.defaultFetchSize = 200                                          |
| 扫描边界   | DynamoDB scan-limit = 10000                                             |
| 并发控制   | 乐观锁重试 4 次 / 间隔 500ms ( @EnableRetry )                                   |
| 数据同步   | DynamoDB Stream / SQS → 14 个 Lambda → MySQL                             |
| CSV 导出 | SuperCSV 注解, ACT/AIR/HTL 三类成本文件                                         |
| 主数据同步  | ImportMdmJob 定时从 MDM API 拉取                                             |
| 实体组织   | 独立 source set src/main/entity                                           |
| 通用查询   | GenericQueryExecutionJob + MGenericQuery* 表 (元数据驱动)                     |
| 环境隔离   | 表名带环境前缀, 「1 账户 1 集群」                                                    |

💡 「元数据驱动的通用查询」是个隐藏亮点： GenericQueryExecutionJob 配合 MGenericQuery\* 表，把查询逻辑外置为元数据，新增查询无需改代码。写「数据治理」「架构可扩展性」时非常好用。

## 五、可用难点素材（按适配度排序）

1. ⑤ **DynamoDB 单表多实体 + 混合存储** ★★★ —— 本主题首选
2. ④ **MySQL 读写分离双数据源** ★★★ —— 数据访问优化
3. ⑧ **跨国多时区一致性** ★★★ —— 特色，强烈建议写
4. ⑥ **Serverless 数据同步与最终一致性** ★★★ —— 一致性保障
5. ⑩ **MDM 主数据统一** ★★ —— 数据治理

## 六、踩坑提醒

| 坑                       | 后果     | 正确做法                                          |
|-------------------------|--------|-----------------------------------------------|
| 只说「用了 MySQL 和 DynamoDB」 | 没有选型逻辑 | 必须写 <b>为什么</b> ：哪类数据适合哪种存储，判据是什么              |
| 把 NoSQL 说成「性能更好」        | 外行表述   | 正确表述是「schema 灵活、可按需扩缩、单分区内查询低延迟」              |
| 回避一致性代价                 | 关键失分   | 明确写出「牺牲强一致换弹性」，并说明为何业务可接受                     |
| 编造分库分表                  | 高风险    | BASE HOME <b>未见分库分表证据</b> ；改写「读写分离双数据源」，同样有分量 |
| 编造 Redis 缓存数据           | 高风险    | 项目 Redis <b>仅用于 Session</b> ，不要写成业务数据缓存       |
| 忽略数据治理                  | 漏考点    | 补上主数据同步、消息 ID 规范、CSV 导出规范等治理手段                |
| 大谈数据湖/湖仓一体              | 无支撑    | 本项目无数据湖，不要硬套                                  |

⚠ **两个技术真实性红线**：① 本项目 **Redis 用于 Session 管理**（`spring-session-data-redis`），未见作为业务数据缓存的证据——写缓存架构时请谨慎，或如实说明「缓存主要用于会话」。② 本项目 **未见分库分表（Sharding）** 证据，数据量级应描述为「通过读写分离与存储选型应对」，而非「分库分表」。

配套：04-高并发与高可用架构.md · 08-批处理架构设计.md（读写分离细节可复用）

## 论文模板 04 · 高并发与高可用架构设计

配套素材见 00-项目素材库.md。本主题几乎年年出现，属必准备项。

### 一、真题常见问法

变体 A（高可用）

请论述你参与的某系统的高可用架构设计，包括冗余、故障转移、容错与降级策略，并说明你如何验证架构的可用性。

变体 B（高性能）

请论述你如何通过架构手段提升系统的并发处理能力与响应性能。

变体 C（可靠性）

请论述你如何保障系统在异常场景下的数据可靠性与业务连续性。

**判断：**BASE HOME 的强项在**变体 C（可靠性）**与**变体 A 的一部分**—— 因为项目有真实的重试、幂等、读写分离、会话外置等机制。变体 B（性能）需要你补充真实压测数据。

二、摘要模板（约 300 字）

本文论述了本人参与的 H.I.S. BASE HOME 旅游业务平台的高可用架构设计与实现。

该平台服务日本 H.I.S. 集团营业本部与旅行者，业务具有强季节性——长假前预约量激增、平峰期骤降，对系统的弹性与可靠性提出很高要求。

本人担任架构师，负责可用性设计、故障隔离方案与可靠性保障机制建设。

针对【在线业务与批量作业资源争抢】、【消息推送重复发送风险】、【多实例部署下的会话一致性】三大挑战，

本文从【资源隔离】、【幂等与重试】、【状态外置】三个维度进行设计：

通过读写分离双数据源隔离批量作业影响，通过幂等时间窗与专用重试服务杜绝重复推送，通过 Redis Cluster 实现会话外置与无状态扩容。

系统上线后【◆ 可用率/故障恢复时间】，验证了方案的可行性。

三、正文五段框架

第 1 段·项目背景与可用性目标（约 400 字）

必须写出「可用性目标」，这是高可用论文的标准起手式。

旅游业务对系统可用性有刚性要求：预约窗口期一旦系统不可用，直接造成订单流失且无法补回。因此我们为该平台定义了明确目标：◆ 年可用率目标（如 99.9%）、◆ 关键接口 P99 响应时间目标、◆ 批处理作业完成时限（如次日营业开始前必须完成）。这些目标驱动了后续所有的架构决策。

本项目背景：

BASE HOME 采用 Java 21 + Spring Boot 3.5.7，部署于 AWS ap-northeast-1，按多模块划分为 Web、Service、Batch、FaaS 等部署单元。本人担任架构师，负责可用性设计与可靠性机制建设。

## 第 2 段 · 架构设计 (约 800 字)

### 第一层：资源隔离——消除单点相互影响

可用性最大的敌人往往不是外部攻击，而是**内部资源争抢**。本项目最典型的场景是：批处理需要从生产库大量读取并回写结果，而在线预约接口同时也在访问同一数据库。若共享连接池，夜间批处理峰值会直接拖慢甚至拖垮在线接口。

我的方案是为批处理配置**双数据源**：`ReaderDataSourceConfig` (`spring.datasource.reader`) 与 `WriterDataSourceConfig` (`spring.datasource.writer`, `@Primary`)，通过包路径隔离 Mapper (`repository/reader/` 与 `repository/writer/`)。读写物理分离后，批量作业不再与在线业务争抢同一数据源连接。

同时设置 `TRANSACTION_READ_COMMITTED` 隔离级别，减少长事务持有锁的时间；`defaultFetchSize = 200` 控制游标，防止海量结果集一次性载入导致内存溢出——这是批处理最常见的 OOM 诱因。

### 第二层：故障隔离——按部署单元切分故障域

Web、Service、Batch、FaaS 各自独立部署：- Batch 模块设置为 `web-application-type: none`，不占用 Web 端口，与 Web 层无任何进程级耦合 - FaaS 侧 14 个 Lambda 按业务域拆分，**单个函数故障不会波及其他业务域** - 每个函数独立镜像、独立流水线、独立部署，支持单函数灰度与快速回滚

**权衡说明**：这增加了运维单元数量与管理复杂度，但把「一个模块崩溃导致全站不可用」的风险降为「单个业务域受影响」。

### 第三层：状态外置——支撑水平扩容

Web 层多实例部署后，若 Session 存于本地内存，负载均衡会把用户请求分发到不同实例，导致反复掉线。我引入 **Spring Session Data Redis**，接入 **Redis Cluster (6 节点)**，客户端使用 Jedis，会话超时设为 3600 秒。会话外置后，Web 实例完全无状态，可任意水平扩容与重启而不影响在线用户。

### 第四层：数据层可靠性

DynamoDB 侧启用乐观锁重试 (`@EnableRetry`，重试 4 次、间隔 500ms)，应对并发写入冲突；文件实体落 S3 而非数据库，降低库体积与备份恢复时长；主数据保留本地只读副本 (26 张 `M*` 表)，使外部 MDM 系统故障不影响业务查询。

## 第 3 段 · 关键技术实现 (约 700 字)

### 要素一：幂等控制——防止重复推送

客户消息与邮件推送是本系统的可靠性敏感点：一旦重复发送，对客户是直接的体验事故。我们设计了多层防护：- **幂等时间窗**：`app.batch.idempotency-duration-minutes = 10`，同一业务键在窗口内重复触发会被拦截 - **专用重试服务**：`MailSendRetryService` 独立承担重试逻辑，与业务处理解耦，避免重试代码侵入业务流程 - **事务粒度拆分**：`service/transaction/*TransactionalService` 将事务边界控制在必要范围内，避免大事务回滚造成的连带重放 - **投递状态跟踪**：`TEmailDeliveryManagement` 表记录投递状态与 `update_travel_status` 函数联动，形成可追溯的闭环

要素二：可观测性——故障可发现才可恢复

高可用的前提是「故障能被及时发现」。Batch 侧通过 **Prometheus PushGateway** 上报作业指标（配置 `shutdown-operation: post`，确保作业结束前完成指标推送），使批处理这类短生命周期进程的运行状态可被监控系统采集。FaaS 侧使用 `aws_lambda_powertools` 的 `Logger` 与 `Tracer` 输出结构化日志与链路追踪信息。

要素三：作业可重跑与可复现

批处理的重跑能力是可靠性的重要保障。我们将 `executionBaseDate`、`acquisitionRange`、`importTargetType` 等关键参数定义为 `JobParameter` 显式传入，不依赖系统时钟，使同一参数的重跑结果完全可复现。◆ 补充：这解决了什么具体的补数场景。

第 4 段 · 难点与解决 (约 700 字)

高可用主题最佳组合：

| 优先级 | 难点                    | 适配理由           |
|-----|-----------------------|----------------|
| 首选  | ⑦ 批量推送的幂等与重试          | 直接对应「可靠性保障」    |
| 次选  | ④ MySQL 读写分离双数据源      | 直接对应「资源隔离」     |
| 三选  | ⑨ 分布式会话 Redis Cluster | 直接对应「无状态扩容」    |
| 加分  | ⑧ 跨国多时区一致性            | 对应「数据可靠性」，特色鲜明 |

难点示例 (⑦，三段式写法)：

**问题：**客户消息常需批量推送（`BulkPutCustomerMessage` 一次处理多条），邮件发送作业在遇到第三方网关限流或网络抖动时必然需要重试。但朴素的重试会引入重复发送——对客户而言这是不可接受的体验事故。这本质上是分布式环境下的**幂等性问题**，而非简单的异常处理问题。

**方案：**我设计了三层防护。第一层是幂等时间窗，以 10 分钟为窗口对同一业务键去重；第二层是把重试能力独立为 `MailSendRetryService`，与业务逻辑解耦，使重试策略可独立调整而无需改动业务流程；第三层是投递状态跟踪表，记录每一次投递的结果，使异常场景下可以精确定位漏发与重发范围。

**效果：**◆ 重复发送率降至 X%，◆ 作业失败可观测并支持定点重跑。

第 5 段 • 总结与不足 (约 200 字)

目前该平台已稳定运行【◆ 时长】，年可用率达到【◆ 数值】，  
批量作业在【◆ 时限】内稳定完成。

不足之处：

一是跨存储的最终一致性链路尚无统一的延迟监控，异常排查路径偏长；

二是 `Lambda` 冷启动在低频函数上仍有可感知延迟。

后续计划补充一致性延迟监控与预置并发配置。

通过本项目我认识到，高可用不是靠单一技术实现的，  
而是由资源隔离、故障隔离、状态外置、幂等重试等一系列设计共同构成的体系。

四、本项目填充点速查

| 论文要素 | 直接可写的内容                                                                             |
|------|-------------------------------------------------------------------------------------|
| 资源隔离 | Batch 读写分离双数据源, Mapper 按 <code>repository/reader</code> / <code>writer</code> 隔离    |
| 事务控制 | <code>TRANSACTION_READ_COMMITTED</code> ; <code>*TransactionalService</code> 拆分事务粒度 |
| 内存保护 | <code>mybatis.defaultFetchSize = 200</code> 控制批量游标                                  |
| 故障隔离 | Web/Service/Batch/FaaS 独立部署; 14 个 Lambda 按业务域拆分                                     |
| 状态外置 | Spring Session Data Redis + Redis Cluster (6 节点) + Jedis, 超时 3600s                  |
| 并发控制 | DynamoDB 乐观锁重试 4 次 / 500ms ( <code>@EnableRetry</code> )                            |
| 幂等   | <code>app.batch.idempotency-duration-minutes = 10</code>                            |
| 重试   | <code>MailSendRetryService</code> 专用重试服务                                            |
| 投递跟踪 | <code>TEmailDeliveryManagement</code> + <code>update_travel_status</code>           |
| 可观测  | Prometheus PushGateway (Batch) ; Powertools Logger/Tracer (FaaS)                    |
| 可复现  | JobParameter 显式传入 <code>executionBaseDate</code> / <code>acquisitionRange</code>    |
| 降级隔离 | 主数据本地只读副本 (26 张 <code>M*</code> 表) , 外部系统故障不影响业务                                    |
| 传输安全 | gRPC 使用 TLS ( <code>tls://</code> endpoint)                                         |

五、可用难点素材 (按适配度排序)

- 1. ⑦ 批量推送的幂等与重试 ★★★★★ —— 本主题首选
- 2. ④ MySQL 读写分离双数据源 ★★★★★ —— 资源隔离
- 3. ⑨ 分布式会话 Redis Cluster ★★★★★ —— 无状态化
- 4. ⑧ 跨国多时区一致性 ★★★★★ —— 数据可靠性 (特色)
- 5. ⑥ Serverless 数据同步 ★★★★★ —— 最终一致性的可靠性设计

## 六、踩坑提醒

| 坑             | 后果  | 正确做法                                                        |
|---------------|-----|-------------------------------------------------------------|
| 只写「用了集群所以高可用」 | 空洞  | 要写可用性目标 → 威胁 → 措施 → 验证这条链                                   |
| 不提可用性指标       | 漏考点 | 必须给出 SLA 目标（◆ 填真实值）与 MTTR                                   |
| 编造双活/异地多活     | 高风险 | 本项目未见多活架构证据；写「多实例 + 状态外置 + 故障域隔离」是真实的                       |
| 编造熔断限流框架      | 高风险 | 本项目未见 Sentinel/Hystrix/Resilience4j 证据，不要写「我们用了熔断降级」        |
| 编造 Redis 业务缓存 | 高风险 | Redis 仅用于 Session，写成「多级缓存架构」会露馅                             |
| 只讲技术不讲验证      | 漏考点 | 补上「如何验证」：TestContainers + LocalStack 做集成验证、Prometheus 做运行监控 |
| 忽略批量作业可靠性     | 片面  | 批处理是可靠性重点，务必写入幂等、重试、可重跑                                     |

⚠ **三条技术真实性红线**（务必遵守）：① **未见熔断/限流组件**（Hystrix、Sentinel、Resilience4j）② **未见 Redis 业务数据缓存**（仅 Session）③ **未见多活/异地容灾架构**

若题目强制要求写熔断降级，可如实写：「当前架构以故障域隔离与幂等重试为主要可靠性手段，熔断降级尚未引入，这是后续改进方向」——**诚实反而加分**。

配套：[03-数据架构与混合存储.md](#) · [08-批处理架构设计.md](#) · [05-系统安全架构.md](#)

## 论文模板 05 · 系统安全架构设计

配套素材见 [00-项目素材库.md](#)。本主题**稳定出现**，且与你的项目契合度高。

### 一、真题常见问法

**变体 A（安全体系整体）**

请论述你参与的某系统的安全架构设计，包括身份认证、访问控制、数据安全与安全审计等方面。

**变体 B（认证授权）**

请论述你如何设计系统的统一身份认证与授权体系，并说明如何实现细粒度权限控制。

变体 C（数据安全）

请论述你如何保障系统敏感数据的机密性、完整性与可用性。

**判断：**BASE HOME 的最强项是**变体 A 与变体 B**——项目真实使用了 OAuth2/OIDC、Spring Security、细粒度 scope 权限、Redis Session、STS 临时凭证、TLS 传输。

二、摘要模板（约 300 字）

本文论述了本人参与的 H.I.S. BASE HOME 旅游业务平台的安全架构设计与实现。该平台承载旅行者的护照信息、亲权同意书（親権同意書）、紧急联系人等敏感个人数据，且需与集团多个外部系统交互，安全要求高、合规约束严。

本人担任架构师，负责身份认证体系设计、权限模型设计与凭证管理方案。针对【多角色细粒度权限控制】、【跨系统凭证泄露风险】、【敏感数据传输与存储安全】三大挑战，

本文从【统一身份认证】、【基于 scope 的细粒度授权】、【临时凭证与加密传输】三个维度进行设计：

基于 Spring Security 集成 OAuth2/OIDC 实现统一登录，以权限 scope 列表驱动接口级授权，以 STS AssumeRole 临时凭证替代长期密钥并与 TLS 结合保障传输安全。

系统上线后【 填效果】，验证了方案的可行性。

三、正文五段框架

第 1 段 • 项目背景与安全需求（约 400 字）

安全论文的开头必须写「保护什么」，否则后面所有措施都失去依据。

BASE HOME 处理的数据具有高度敏感性，我将其分级为三类：① **个人身份信息**——旅行者护照信息、姓名、出生日期；② **特殊敏感数据**——未成年人亲权同意书（親権同意書）、紧急联系人信息；③ **业务敏感数据**——预约成本、客户消息内容。此外，系统需与集团 MDM 主数据平台、外部预约系统、邮件网关、消息推送服务等多方交互，每个交互面都是潜在的攻击入口。合规方面，系统需满足日本个人信息保护法的相关要求。

第 2 段 • 架构设计（约 800 字）

按安全架构的经典四层组织：身份认证 → 访问控制 → 数据安全 → 安全审计

第一层：身份认证（Authentication）

系统采用 **OAuth2/OIDC** 协议实现统一身份认证，Web 层通过 `spring-boot-starter-oauth2-client` 接入集团统一认证服务，以授权码模式完成登录，令牌校验使用 `spring-security-oauth2-jose` (JOSE/JWT)。用户身份属性通过 `user-name-attribute` 配置从令牌中提取，避免依赖本地用户表——这使账号生命周期由集团统一治理。

同时通过 `thymeleaf-extras-springsecurity6` 把安全上下文暴露到视图层，使页面元素的显隐可以直接由权限决定，而不是靠前端 JavaScript 隐藏（后者只是「看不见」，后者才能做到「点不到」）。

**权衡说明：**OIDC 引入了对外部认证服务的依赖，我们通过将用户信息缓存于会话中，使认证服务短暂不可用时已登录用户仍可继续操作，避免认证服务成为可用性单点。

## 第二层：访问控制 (Authorization)

授权模型我采用 **RBAC + 接口级 scope 校验** 的组合。角色维度划分为管理员 (admin)、操作员 (operator)、主管 (chief)、VIP 等，各角色持有不同的权限 scope 列表。权限校验下沉到接口层 (`/api/v1/**` 由 Spring Security 统一拦截)，而非散落在各 Controller 的业务代码中——这样新增接口时权限是「默认拒绝」而非「默认放行」，从架构上避免了「忘记加权限注解」这类遗漏风险。

## 第三层：数据安全 (Data Security)

**传输安全：**内部服务间 gRPC 调用使用 TLS (endpoint 采用 `tls://` 前缀)，对外 Web 访问全站 HTTPS。**凭证安全：**这是我重点处理的部分。系统不使用长期固定的 AWS 密钥，而是通过 **STS AssumeRole 获取临时凭证**，配合 `DefaultCredentialsProvider` 兜底，凭证自动轮换，即使泄露其有效期也极短。**配置安全：**FaaS 侧所有配置（数据库连接、外部端点）统一从 SSM Parameter Store 读取，代码仓库中不落任何明文凭证。**存储安全：**文件实体统一存放 S3，数据库仅保存文件引用；数据库账号按 reader/writer 职责分离，批处理读账号无写权限。

## 第四层：安全审计 (Audit)

系统在 Web 层设置独立的审计能力：画面操作日志由 `system` 域的控制器的统一记录，关键业务操作（预约变更、消息发送、主数据修改）均留存操作者、操作时间与操作内容。审计日志与业务数据分离存储，避免业务回滚时日志同时丢失。

## 第 3 段 • 关键技术实现 (约 700 字)

### 要素一：会话安全

会话由 **Spring Session Data Redis** 统一管理，存储于 **Redis Cluster (6 节点)**，超时时间 3600 秒。相比本地会话，集中式会话带来两个安全收益：一是会话数据不落在应用服务器磁盘上，降低主机被入侵后的会话窃取风险；二是可在服务端强制失效任意会话，实现「管理员踢出用户」与「用户改密后全端下线」这类安全能力。

要素二：凭证与密钥的集中治理

我把凭证获取统一收口到配置类（`DynamoDbConfig`、`AwsConfig`、`SsmClient`），业务代码不直接接触凭证。这样做的价值在于：凭证轮换、加密算法升级、权限范围调整都只需在单一位置修改，不会遗漏调用点。环境隔离同样重要——DynamoDB 表名统一带环境前缀，实现「1 账户 1 集群」，防止测试环境误连生产数据。

要素三：接口契约中的安全边界

由于系统通过 `protobuf` 向 Java/Go/PHP 三端提供接口，契约本身也承担安全职责：每个 RPC 均声明 HTTP 映射（`google.api.http`），统一走 `/v1/<RpcName>` 路径，便于在网关层集中做鉴权与限流；错误信息通过统一的 `Error{code,message}` 结构返回，对外只暴露错误码与友好提示，**不泄漏堆栈与内部实现细节**。

要素四：输入校验与注入防护

Web 层启用 `spring-boot-starter-validation` 做参数校验，数据库访问全部通过 MyBatis 参数绑定，杜绝字符串拼接 SQL。涉及 HTML 内容处理的场景使用 Jsoup 做净化，防止存储型 XSS——这在客户消息场景尤其重要，因为消息内容会在 MyPage 页面回显给其他用户。

第 4 段 • 难点与解决（约 700 字）

安全主题最佳组合（安全主题的难点需要自行组织，素材库难点适配度中等）：

| 优先级 | 难点方向            | 素材来源                        |
|-----|-----------------|-----------------------------|
| 首选  | 多角色细粒度权限的默认拒绝设计 | 由项目 scope 权限模型推导            |
| 次选  | 三语言契约下的安全边界统一   | 素材库 ①                       |
| 三选  | 跨国多时区与数据合规      | 素材库 ⑧                       |
| 加分  | 凭证长期化风险治理       | 素材库 ③（v1/v2 SDK 并存背景下的凭证统一） |

难点示例（权限默认拒绝）：

**问题：**系统角色众多（管理员、操作员、主管、VIP 等），且不同角色可见的业务域差异极大。项目初期采用在 Controller 方法上 逐个添加权限注解的方式，随着接口数量增长（REST 接口按业务域 分布在 8 个包下），出现了「新增接口忘记加注解」的遗漏风险——这类漏洞不会在测试期暴露，只在被利用时才显现。

**方案：**我把授权策略从「逐个放行」改为「默认拒绝」。在 Spring Security 配置中统一拦截 `/api/v1/**`，未显式声明的路径一律拒绝；权限以 scope 列表的形式 随令牌下发，接口层按 scope 校验而非按角色硬编码判断。这样角色与权限的关系可以在配置侧调整，无需修改业务代码——新增角色只需分配 scope 组合。

**效果：**◆ 消除新增接口的权限遗漏风险；角色调整从「改代码」变为「改配置」，◆ 权限变更耗时从 X 降到 Y。

第 5 段 • 总结与不足 (约 200 字)

目前该安全架构已稳定支撑【◆ 用户规模】，未发生【◆ 安全事件情况】。

不足之处：

一是权限模型目前以角色为单位的 `scope` 组合为主，

尚未实现对单条数据级（行级）的细粒度控制；

二是跨语言契约的破坏性变更尚未引入自动化安全影响评估。

后续计划引入数据级授权与契约变更审计。

通过本项目我认识到，安全架构的核心不是堆砌安全产品，

而是建立“默认拒绝”的基线，并让凭证、权限、审计都收口到可治理的位置。

四、本项目填充点速查

| 论文要素  | 直接可写的内容                                                                                   |
|-------|-------------------------------------------------------------------------------------------|
| 认证协议  | OAuth2 / OIDC 授权码模式                                                                       |
| 认证实现  | <code>spring-boot-starter-oauth2-client</code> + <code>spring-security-oauth2-jose</code> |
| 身份来源  | 令牌中的 <code>user-name-attribute</code> ，账号由集团统一治理                                          |
| 视图层安全 | <code>thymeleaf-extras-springsecurity6</code> （页面元素按权限显隐）                                 |
| 授权模型  | 角色 + scope 权限列表；接口层统一拦截 <code>/api/v1/**</code>                                           |
| 角色划分  | admin / operator1~4 / chief / vip                                                         |
| 会话管理  | Spring Session Data Redis + Redis Cluster（6 节点）+ Jedis，3600s                              |
| 传输安全  | gRPC 使用 <code>tls://</code> endpoint；对外 HTTPS                                             |
| 凭证管理  | STS AssumeRole 临时凭证 + <code>DefaultCredentialsProvider</code>                             |
| 配置安全  | SSM Parameter Store（FaaS）；凭证集中在 <code>DynamoDbConfig</code> / <code>AwsConfig</code>      |

| 论文要素  | 直接可写的内容                                                               |
|-------|-----------------------------------------------------------------------|
| 环境隔离  | DynamoDB 表名带环境前缀，「1 账户 1 集群」                                          |
| 数据库权限 | reader / writer 双数据源职责分离                                              |
| 审计    | <code>system</code> 域画面操作日志（操作者/时间/内容）                                |
| 输入校验  | <code>spring-boot-starter-validation</code> ；MyBatis 参数绑定防注入          |
| 内容净化  | Jsoup (1.22.1) 处理 HTML 内容，防存储型 XSS                                    |
| 错误信息  | 统一 <code>Error{code,message}</code> ，不泄漏内部细节                          |
| 消息规范  | 消息 ID 前缀区分类型（E=错误、F=字段标签）                                             |
| 接口文档  | SpringDoc OpenAPI <code>/api-docs</code> ，仅匹配 <code>/api/v1/**</code> |
| 日志脱敏  | properties 自动生成消息常量，避免硬编码敏感文案                                         |

💡「消息 ID 体系」是一个容易被忽略的亮点：系统通过 properties 文件自动生成消息常量类（`ApiMessageId`、`WebMessageId`、`FormItem`，均标注「自动生成、修正不可」），并以 ID 首字符区分消息类型。这既是代码规范，也是安全审计与国际化的双重基础设施——写安全或可维护性时都可以用。

## 五、可用难点素材（按适配度排序）

- 1. 权限模型的「默认拒绝」重构 ★★ ★ —— 本主题首选（需自行组织，参考上文示例）
- 2. ① 一份 proto 契约驱动三语言 ★★ —— 契约级安全边界统一
- 3. ③ AWS SDK v1/v2 并存下的凭证统一 ★★ —— 凭证治理
- 4. ⑧ 跨国多时区一致性 ★★ —— 数据合规维度
- 5. ⑨ 分布式会话 Redis Cluster ★★ —— 会话安全

## 六、踩坑提醒

| 坑              | 后果   | 正确做法                                      |
|----------------|------|-------------------------------------------|
| 罗列安全产品         | 空洞   | 按「保护什么 → 威胁是什么 → 措施是什么」组织                 |
| 编造等保/ISO 认证    | 高风险  | 未确认的合规资质不要写；写「参照日本个人信息保护法要求」更稳妥           |
| 编造加密算法细节       | 高风险  | 只写「TLS 传输加密」「STS 临时凭证」，不编 AES-256/国密等具体算法 |
| 编造 WAF/堡垒机/IDS | 高风险  | 本项目未见此类证据，不要写                             |
| 忽略审计           | 漏考点  | 安全架构四要素「认证/授权/加密/审计」缺一不可                  |
| 只说技术不说管理       | 片面   | 补上凭证轮换、环境隔离、权限变更流程等治理手段                   |
| 忘记「默认拒绝」原则     | 关键失分 | 这是安全架构最重要的设计原则，务必体现                       |

⚠️ **技术真实性提醒：** 本项目有 OAuth2/OIDC + Spring Security + TLS + STS 临时凭证 + Jsoup 净化，但未见 WAF、IDS/IPS、堡垒机、加密机、国密算法等企业级安全设备的使用证据。论文中不要为了「显得安全」而堆砌未使用的产品。相反，把「默认拒绝的授权设计」「凭证集中治理」这两个**真实的架构决策**写透，比列十个安全产品更有说服力。

配套：01- 微服务架构设计.md · 06- 系统集成与gRPC.md (契约安全部分)

## 论文模板 06 · 系统集成与异构系统互操作 (gRPC / 契约驱动)

配套素材见 00- 项目素材库.md 。本主题是 BASE HOME 的绝对强项——「一份 proto 驱动 Java/Go/PHP 三语言」是极少有人能写出的真实素材，竞赛压力小，**强烈建议重点准备。**

### 一、真题常见问法

#### 变体 A (系统集成整体)

请论述你参与的某系统的集成方案设计，包括集成方式选型、接口契约管理与异构系统互操作等方面。

#### 变体 B (接口/契约管理)

请论述你如何在多团队、多技术栈环境下保证接口契约的一致性。

#### 变体 C (企业应用集成 EAI)

请论述你如何实现企业内多个异构系统之间的数据集成与流程集成。

**判断：**变体 B 是本项目的**最优命中区**；变体 A、C 也可用同一套素材，只需调整论述重心。

## 二、摘要模板 (约 300 字)

本文论述了本人参与的 H.I.S. BASE HOME 旅游业务平台的系统集成架构设计与实现。该平台需同时为 Java 内部服务、Go 周边平台与 PHP 既有系统三类调用方提供服务，并与集团 MDM 主数据平台、外部预约系统、邮件与消息网关等多方交互。本人担任架构师，负责接口契约设计、异构系统集成方案与集成治理机制建设。针对【多技术栈下接口定义易漂移】、【依赖版本隐式解析引发运行时故障】、【外部系统故障影响业务连续性】三大挑战，本文从【契约单一来源】、【构建期版本治理】、【外部依赖本地化】三个维度进行设计：以 Protocol Buffers 作为唯一契约源，通过构建流水线生成 Java、Go、PHP 三语言产物及 OpenAPI 文档；以统一版本管理插件锁定关键依赖；以本地只读主数据副本隔离外部平台故障。系统上线后【 填效果】，验证了方案的可行性。

## 三、正文五段框架

### 第 1 段·项目背景与集成挑战 (约 400 字)

集成类论文的开头要写清「谁和谁集成」。

BASE HOME 处于集团 IT 版图的中位位置，集成关系复杂：- **上游**：集团 MDM 主数据平台（本国、城市、航空公司、货币等主数据）- **对外**：周边平台通过 Go 客户端接入，既有系统通过 PHP 客户端接入 - **对内**：Web 层通过 gRPC 调用 Service 层（17 个 RPC）- **下游**：邮件网关、SNS 消息推送、SQS 队列、DynamoDB Stream

多技术栈并存带来一个根本性问题：**接口定义在哪里是「真」的？** 如果每个调用方各自维护一份接口定义，字段口径必然随时间漂移，而这类问题往往在联调甚至上线后才暴露。

### 第 2 段·架构设计 (约 800 字，本文核心)

#### 第一层：契约单一来源 (Single Source of Truth)

我把 `base-home-spec` 模块确立为**唯一的接口契约源**，所有接口以 protobuf 定义，统一存放于 `his/base/home/v1/` 目录，按职责分层组织：- `basehome_service.proto` —— 服务定义（`BaseHomeService`，17 个 RPC）- `request/` —— 17 个请求消息 - `response/` —— 17 个响应消息 - `enums/` —— 13 个枚举（产品类型、文档类型、申请人类型、消息状态、销售渠道、发送者类型、保险类型等）- `type/` —— 约 42 个可复用消息（含统一的 `Error{code,message}`）

全部约 90 个 proto 文件纳入版本管理，任何接口变更都必须通过该模块提交，从流程上杜绝了「各写一份」的可能。

**层次化组织的价值**：`type/` 中的可复用消息（如地址、金额、旅行者基础信息）被多个 request/response 引用，避免了同一结构在 17 个接口中重复定义 17 次——这是契约可维护性的关键。

第二层：构建期多语言产物生成

契约定义统一后，我在构建脚本中编排了完整的生成链路：`build.sh` 按 `build-java` → `build-go` → `build-php` → `generate-all` 顺序执行，每种语言使用各自的官方插件：

| 目标语言 | 生成方式                                                                                                             | 产物形态          |
|------|------------------------------------------------------------------------------------------------------------------|---------------|
| Java | <code>protobuf-gradle-plugin</code> ( <code>protoc</code> + <code>protoc-gen-grpc-java</code> )                  | 发布至内部 Maven   |
| Go   | 容器内 <code>protoc</code> ( <code>--go_out</code> / <code>--go-grpc_out</code> / <code>--grpc-gateway_out</code> ) | Go module zip |
| PHP  | <code>--php_out</code> + <code>grpc_php_plugin</code>                                                            | Composer 包    |
| 文档   | <code>--openapiv2_out</code> → MkDocs                                                                            | 在线 API 文档     |

**关键设计：**把「生成」纳入构建流水线而非依赖人工执行，使接口一致性问题从「运行时联调发现」提前到「构建期暴露」——这是本次集成设计最重要的收益。

**权衡说明：**多语言生成的构建链路复杂度显著上升（需维护 Docker 化的 `protoc` 环境、三套发布流程），但相比接口不一致导致的线上缺陷与联调返工，这个成本是值得的。

第三层：对外依赖本地化

集团 MDM 主数据平台是外部系统，若业务查询直连外部，则外部系统的任何抖动都会直接传导到业务操作。我采用**本地只读副本**策略：通过 `ImportMdmJob` 定时从 MDM API 拉取主数据，落地为 26 张 `M*` 主数据表，业务侧只查本地。这样 MDM 平台短暂不可用时业务不受影响，数据新鲜度通过同步频率控制，是「可用性优先」的合理取舍。

第 3 段 • 关键技术实现 (约 700 字)

要素一：通信协议选型

服务间通信选择 **gRPC** 而非 REST，依据有三：① HTTP/2 多路复用，在高频内部调用下连接利用率更高；② `protobuf` 二进制编码体积小、序列化开销低；③ **强类型契约天然由 `proto` 保证**，与我们的契约单一来源策略一致。服务端监听 6565 端口，单次消息上限设为 64MB（旅游业务存在较大报文，如批量消息与文件列表），并开启 `gRPC Reflection` 便于调试期接口探查。

对外则通过 Web 层的 REST 接口（`/api/v1/**`）暴露，兼顾浏览器友好性与前端契约稳定性，并用 `SpringDoc` 生成 OpenAPI 描述（`/api-docs`）。

要素二：服务端的分发设计

`base-home-service` 以单一入口类 `BaseHomeService`（`@GRpcService`，继承 `proto` 生成的 `BaseHomeServiceImplBase`）承接 17 个 RPC，统一经 `execute(IGrpcService, req, respObserver)` 方法分发。业务逻辑委托给 18 个实现类（均实现 `IGrpcService` 接口）。这一设计的价值在于：**横切关注点集中**——日志、异常转换、认证上下文校验都可在 `execute` 中统一处理，各业务实现类只关注业务逻辑，避免每个 RPC 重复编写样板代码。

**权衡说明：**单一入口在 RPC 数量极大时会成为维护瓶颈，但 17 个 RPC 的规模下，统一分发带来的横切收益大于其复杂度成本。

要素三：跨业务域能力复用

多个 RPC 需要的能力（文档、消息线程、亲权同意书、序列号、旅行信息）下沉到 `service/shared/` 中的 5 个服务，避免同一逻辑在多个 RPC 中重复实现。文件相关能力单独归入 `service/filesystem/`。

要素四：错误契约统一

`proto` 中定义统一的 `Error{code, message}` 消息，所有接口的错误返回结构一致，调用方可编写统一的错误处理逻辑，无需为每个接口单独适配。错误码体系与 `his-common` 的消息 ID 规范对齐（如 `EABH00001` 形式的错误码），使错误在服务端、Web 端与文档中具有一致的标识。

第 4 段 • 难点与解决 (约 700 字)

集成主题最佳组合：

| 优先级 | 难点                   | 适配理由         |
|-----|----------------------|--------------|
| 首选  | ① 一份 proto 契约驱动三语言   | 主题核心，独一无二    |
| 次选  | ② protobuf 版本冲突致启动失败 | 直接对应「集成治理」   |
| 三选  | ③ AWS SDK v1/v2 并存   | 直接对应「异构共存」   |
| 加分  | ⑩ MDM 主数据统一          | 直接对应「企业应用集成」 |

难点示例（②，三段式）：

**问题：** 契约统一后，新的问题出现了—— 契约的「消费者」不止我们的服务，还包括 protobuf 官方的公共类型库。一次依赖升级中， `proto-google-common-protos` 被解析到 2.64.0-rc1， gRPC 服务启动直接失败。更棘手的是，由于我们的版本声明采用了 动态区间 ( `protobufVersion = '[3,4]'`、 `grpcVersion = 'latest.release'` )， 这个问题在构建期完全不显现，只在运行时暴露。

我的判断是：这不是偶发故障，而是**契约不完整**的表现—— 我们统一了业务接口的契约，却没有统一「第三方依赖版本」这一隐式契约。 因此我没有做局部修复，而是在 `base-home-versions` 统一版本源中 用 `resolutionStrategy { force ... }` 将 `proto-google-common-protos` 锁定到 2.63.2，并同步锁定 MyBatis 3.0.5 等关键组件。

**效果：** 启动失败问题消除；同一提交在任何机器上构建产物一致， **构建可复现性**得到保障。

第 5 段 • 总结与不足 (约 200 字)

目前该集成架构已稳定支撑 Java、Go、PHP 三端调用，  
17 个 RPC 在三端口径一致，接口文档随契约自动生成。

不足之处：

一是跨语言契约虽来源统一，但\*\*破坏性变更尚未实现自动化的兼容性检测\*\*，  
目前依赖人工评审；

二是 Go 与 PHP 侧的对齐仍依赖固定版本号人工同步，  
尚未做到发布即对齐。

后续计划引入契约兼容性校验与三端发布联动。

通过本项目我认识到，系统集成的本质不是"把系统连起来"，  
而是"让多个系统对同一件事保持一致的理解"。

四、本项目填充点速查

| 论文要素    | 直接可写的内容                                                                                                         |
|---------|-----------------------------------------------------------------------------------------------------------------|
| 契约源     | <code>base-home-spec</code> ，protobuf 唯一契约源，约 90 个 proto 文件                                                     |
| 服务定义    | <code>basehome_service.proto</code> → <code>service BaseHomeService</code> ，17 个 RPC                            |
| 契约分层    | <code>request/</code> (17) + <code>response/</code> (17) + <code>enums/</code> (13) + <code>type/</code> (约 42) |
| 统一错误    | <code>type/error.proto</code> → <code>Error{code, message}</code>                                               |
| 多语言生成   | Java (protobuf-gradle-plugin) / Go (protoc + grpc-gateway) / PHP (grpc_php_plugin)                              |
| 生成编排    | <code>build.sh</code> ：build-java → build-go → build-php → generate-all                                         |
| 文档生成    | <code>--openapi2_out</code> → <code>base_home_api.swagger.json</code> → MkDocs 站点                               |
| HTTP 映射 | 每个 RPC 带 <code>google.api.http</code> 映射，路径 <code>/v1/&lt;RpcName&gt;</code>                                    |
| 服务通信    | gRPC 端口 6565， <code>maxInboundMessageSize</code> 64MB，开启 Reflection                                             |
| 服务端设计   | 单入口 <code>BaseHomeService</code> + <code>execute()</code> 统一分发，18 个实现类                                          |

| 论文要素   | 直接可写的内容                                                                       |
|--------|-------------------------------------------------------------------------------|
| 能力复用   | <code>service/shared/</code> 5 个共享服务; <code>service/filesystem/</code> 2 个    |
| 调用方式   | 配置化 endpoint ( <code>tls://</code> ) + TLS 传输加密                               |
| 对外接口   | Web 层 REST <code>/api/v1/**</code> , SpringDoc OpenAPI <code>/api-docs</code> |
| 主数据集成  | <code>ImportMdmJob</code> 定时从 MDM API 同步, 本地 26 张 <code>M*</code> 只读副本        |
| 外部预约集成 | <code>ExternalReservationImportJob</code> + <code>RecvImportType</code>       |
| 异步集成   | SNS 主题、SQS 队列、DynamoDB Stream → Lambda                                        |
| 版本治理   | <code>base-home-versions</code> 插件 + <code>resolutionStrategy force</code>    |
| 版本对齐   | <code>his-common-spec</code> 2.6.0 在 Java/Go/PHP 三侧对齐固定                       |
| 依赖方向   | web/service → common + spec; batch → common + mdm-spec                        |
| 文档发布   | MkDocs 站点 (容器化, 端口 8888)                                                      |

## 五、可用难点素材（按适配度排序）

- 1. ① 一份 proto 契约驱动三语言 ★★★★★ —— 本主题核心
- 2. ② protobuf 版本冲突致启动失败 ★★★★★ —— 集成治理
- 3. ③ AWS SDK v1/v2 并存 ★★★★★ —— 异构共存与渐进演进
- 4. ⑩ MDM 主数据统一 ★★★★★ —— 企业应用集成
- 5. ⑥ Serverless 数据同步 ★★ —— 异步集成

## 六、踩坑提醒

| 坑           | 后果     | 正确做法                                                    |
|-------------|--------|---------------------------------------------------------|
| 只说「用了 gRPC」 | 无深度    | 必须写 <b>为什么不用 REST</b> (HTTP/2、强类型契约、二进制编码效率)            |
| 不写契约管理      | 关键失分   | 「契约单一来源」是本主题的灵魂, 务必写透                                   |
| 编造服务注册中心    | 高风险    | 本项目 gRPC 为 <b>endpoint 配置直连</b> , 无 Eureka/Nacos/Consul |
| 编造 API 网关   | 高风险    | 本项目未见网关组件, 不要写「通过网关统一鉴权」; 可写「统一路径便于网关层集中治理」作为设计意图       |
| 编造 ESB      | 高风险    | 本项目未用 ESB, 集成靠契约 + 消息队列, 如实写即可                          |
| 忽略版本治理      | 漏考点    | 集成不仅是「连起来」, 还包括依赖版本的一致性                                 |
| 忘记写权衡       | 缺乏架构思维 | 多语言生成的构建复杂度 vs 接口不一致的返工成本, 要明确对比                        |

💡 **本主题的独特优势：**「一份契约生成 Java/Go/PHP 三语言 + 自动生成 OpenAPI 文档」这个素材**绝大多数考生写不出来**，因为它需要真实的多语言项目经历。如果你的考题涉及「接口管理」「多团队协作」「异构系统集成」，优先选这道题，用 ① 和 ② 两个难点撑满「难点」段，基本可以保证在这一题上拿到高于平均的分数。

配套：01- 微服务架构设计.md (服务间通信) · 03- 数据架构与混合存储.md (数据集成)

## 论文模板 07 · 架构评估与质量属性

配套素材见 00-项目素材库.md。本主题**理论性最强**，也是最容易写出「架构味」的一类题。⚠ 本主题要求掌握 ATAM / 质量属性场景等**教材概念**，请先复习教材对应章节。

### 一、真题常见问法

#### 变体 A (架构评估)

请论述你如何对系统架构进行评估，包括评估方法、评估过程与评估结果的应用。

#### 变体 B (质量属性)

请论述质量属性在架构设计中的作用，并结合项目说明你如何实现 性能、可用性、安全性等质量属性之间的权衡。

#### 变体 C (架构决策)

请论述你在架构设计过程中如何进行技术决策，如何处理多个质量属性之间的冲突。

**判断：**变体 B 是 BASE HOME 的最优命中区——项目有大量真实的「权衡决策」可写（混合存储的取舍、Serverless 的一致性取舍、双 SDK 并存的取舍）。

### 二、必背理论要点（先背熟，再谈项目）

#### 2.1 六大质量属性

| 属性  | 含义         | 本项目对应设计                                        |
|-----|------------|------------------------------------------------|
| 性能  | 响应时间、吞吐量   | gRPC (HTTP/2)、读写分离、fetchSize 控制、DynamoDB 单分区查询 |
| 可用性 | 系统正常运行时间比例 | 故障域隔离、会话外置、乐观锁重试、幂等                            |
| 安全性 | 抵御攻击、保护数据  | OAuth2/OIDC、默认拒绝授权、TLS、STS 临时凭证                |

| 属性   | 含义        | 本项目对应设计                               |
|------|-----------|---------------------------------------|
| 可修改性 | 变更的成本与难度  | 契约单一来源、版本统一治理、模块化拆分                   |
| 可测试性 | 发现缺陷的难易   | TestContainers、LocalStack、pytest+moto |
| 易用性  | 用户完成任务的效率 | MyPage 界面、ToDo 管理、通知推送                |

2.2 质量属性场景六要素



示例（本项目可用）：

当营业本部操作员（刺激源）在长假前预约高峰期（环境）提交**预约查询请求**（刺激）时，**Web 与 Service 模块**（制品）应在**2 秒内**（响应度量）返回结果（响应）。

2.3 ATAM 四个阶段与关键概念

ATAM 四阶段：

- 1. 场景和需求收集 —— 与利益相关者共同梳理质量属性场景
- 2. 架构视图和场景实现 —— 描述架构如何支持各场景
- 3. 属性模型构造和分析 —— 建立性能/可用性等分析模型
- 4. 架构决策与折中 —— 识别权衡点并形成决策

四个关键概念（必考，务必准确使用）：

| 概念   | 定义                       | 本项目示例                                                               |
|------|--------------------------|---------------------------------------------------------------------|
| 敏感点  | 为实现某质量属性，一个或多个构件所具有的关键性质 | DynamoDB 的 <code>travel_no</code> 分区键设计是 <b>性能</b> 的敏感点（决定查询是否落单分区） |
| 权衡点  | 影响多个质量属性、且对多个属性敏感的敏感点    | <b>混合存储选型</b> ：提升性能与可扩展性（性能/可修改性），但增加运维复杂度（可维护性/可测试性）               |
| 风险点  | 可能导致问题的架构决策              | 采用动态版本区间（ <code>latest.release</code> ）——导致构建不可复现（已发生实际故障）          |
| 非风险点 | 经分析确认安全的决策               | 使用 Spring Session + Redis Cluster 实现会话外置——已有成熟实践支撑                  |

💡 考场上「敏感点/权衡点/风险点」这三个词只要用对，就能明显区分于普通考生。建议在「难点」段中显式使用这些术语。

2.4 常见质量属性策略（战术）

| 属性   | 常见策略                 |
|------|----------------------|
| 可用性  | 冗余、故障检测、恢复、故障隔离      |
| 性能   | 资源调度、并发、减少计算/存储开销、缓存 |
| 可修改性 | 限制依赖、抽象、信息隐藏、延迟绑定    |
| 安全性  | 认证、授权、限制访问、审计        |
| 可测试性 | 记录回放、接口分离、可观测        |

三、正文五段框架

第 1 段 • 项目背景与质量属性需求（约 400 字）

BASE HOME 是 H.I.S. 面向旅游业务的支撑平台，采用 Java 21 + Spring Boot 3.5.7，部署于 AWS ap-northeast-1。本人担任架构师，负责架构设计与质量属性权衡。

在项目初期，我与业务方、运维方共同梳理了关键质量属性需求，并 **将其转化为可度量的质量属性场景**，作为后续所有架构决策的依据：

| 质量属性 | 场景（简化）              | 度量目标              |
|------|---------------------|-------------------|
| 可用性  | 长假预约高峰期间，Web 层单实例故障 | ◆ 服务不中断，用户会话不丢失   |
| 性能   | 高峰期操作员提交预约查询        | ◆ $P99 \leq 2$ 秒  |
| 安全性  | 未授权用户尝试访问客户消息接口     | 拒绝访问并记录审计日志       |
| 可修改性 | 新增一类通知渠道            | ◆ 不改动核心服务，仅新增一个单元 |
| 可测试性 | 开发人员本地验证数据同步逻辑      | 无需连接真实 AWS 即可完成   |

这种「场景化」的表达方式，使质量属性从抽象口号变成了可验证的工程目标。

⚠ **注意：**表格中的 ◆ 数字必须由你填真实值。但即使填了估计值，「把需求写成场景」这一动作本身就体现架构思维。

第 2 段 • 评估方法与过程（约 800 字，本文核心）

推荐用 ATAM 四阶段组织：

阶段一：场景和需求收集

我组织了由业务方（营业本部代表）、运维方、开发负责人参与的评估会议，采用 **效用树（Utility Tree）** 对质量属性需求进行优先级排序。排序依据为「重要性」与「实现难度」两个维度，高重要性且高难度的需求优先投入架构资源。经排序后，**性能与可用性**被列为最高优先级，**可修改性**次之（因业务政策变动频繁，接口与字段需持续演进）。

## 阶段二：架构视图和场景实现

我以视图为单位描述架构对场景的支撑，本项目主要涉及四个视图：- **逻辑视图**：8 个 Gradle 模块的职责划分与依赖方向 - **开发视图**：`common`（实体/常量/消息）、`spec`（契约）、`versions`（版本治理）等支撑模块的组织 - **进程视图**：Web（8080）、Service（gRPC 6565）、Batch（无 Web 容器）、FaaS（14 个 Lambda）的运行时交互 - **部署视图**：AWS ap-northeast-1，多实例部署 + Redis Cluster + Aurora + DynamoDB + S3 + SQS/SNS

## 阶段三：属性模型构造和分析

针对性能与可用性，我们构造了简化的分析模型：- **性能**：分析关键路径（Web → gRPC → DynamoDB），识别出 DynamoDB 查询是否落在单一分区是主导因素，因此把分区键设计作为性能敏感点重点评审 - **可用性**：分析方法是部署单元逐个标记为「失效」，观察影响范围。分析中发现 Batch 与 Web 共享数据源时，批处理峰值会传导为在线接口延迟——这成为后续读写分离的直接依据

## 阶段四：架构决策与折中

评估的核心产出是**识别权衡点并形成决策记录**。本项目的三个主要权衡：

**权衡一：混合存储（MySQL + DynamoDB + S3）** - 正向：性能↑（DynamoDB 低延迟、单分区查询）、可修改性↑（schema-less，字段演进免 DDL） - 负向：可维护性↓（多套存储的备份/容量规划）、一致性↓（需事件驱动达成最终一致） - **决策**：接受。理由是四类数据的访问特征差异足够大，统一存储的代价更高。

**权衡二：Serverless 数据同步** - 正向：性能↑（主链路解耦）、成本↓（按需计费）、可用性↑（故障域隔离） - 负向：一致性↓（最终一致）、可测试性↓（本地调试困难） - **决策**：接受。理由是同步场景对延迟不敏感，且用 moto/LocalStack 缓解了测试困难。

**权衡三：契约单一来源** - 正向：可修改性↑（接口变更一处生效）、可测试性↑（构建期暴露不一致） - 负向：构建复杂度↑（需维护 Docker 化 protoc 环境与三套发布流程） - **决策**：接受。理由见难点素材 ①。

## 第 3 段 • 敏感点、风险点识别 (约 700 字)

这一节是本文的加分核心，务必显式使用术语。

敏感点识别：

通过评估，我们识别出以下关键敏感点：

| 敏感点                                                                | 影响的质量属性   | 说明                     |
|--------------------------------------------------------------------|-----------|------------------------|
| DynamoDB 分区键设计 ( <code>travel_no</code> + <code>data_type</code> ) | 性能        | 决定查询能否落单分区，是延迟的主导因素    |
| <code>mybatis.defaultFetchSize = 200</code>                        | 性能、可用性    | 批量游标大小直接关系内存占用与 OOM 风险 |
| Session 存储位置 (本地 vs Redis)                                         | 可用性       | 决定 Web 实例能否水平扩容与无损重启   |
| 依赖版本声明方式 (区间 vs 固定)                                                | 可修改性、可测试性 | 决定构建是否可复现              |
| 授权校验位置 (接口层 vs 业务代码)                                               | 安全性       | 决定权限遗漏风险的高低            |

风险点与非风险点：

**风险点：**采用 `[3,4)` 与 `latest.release` 这样的动态版本区间，使构建产物不可复现，任何一次依赖发布都可能引入破坏性变更。该风险在评估中被识别为高风险——**并且后续真实发生了**（`proto-google-common-protos 2.64.0-rc1` 导致服务启动失败）。这印证了评估的价值：被识别的风险可以提前预案。

**非风险点：**Spring Session + Redis Cluster 的会话外置方案。该方案在业界有大量成熟实践，且 Redis Cluster 本身提供分片与故障转移，评估确认无需额外预案。

第 4 段 • 难点与解决 (约 700 字)

架构评估主题最佳组合：

| 优先级 | 难点                           | 适配理由          |
|-----|------------------------------|---------------|
| 首选  | ② protobuf 版本冲突 (风险点转化为实际故障) | 对应「风险点识别」     |
| 次选  | ⑤ 混合存储 (典型权衡点)               | 对应「权衡点分析」     |
| 三选  | ⑥ Serverless 同步 (一致性与性能的权衡)  | 对应「属性冲突处理」    |
| 加分  | ③ AWS SDK v1/v2 并存           | 对应「可修改性的长期成本」 |

**难点写法提示：**本主题的难点要用质量属性的语言写，而不是用技术语言。例如不要写「我们把版本锁住了」，而要写「我们识别出该决策对可修改性与可测试性构成风险，通过将隐式版本契约显式化，把风险转化为可控状态」。

第 5 段 • 总结与不足 (约 200 字)

通过本次评估，我们共识别出【◆ 数量】个敏感点、【◆ 数量】个权衡点与【◆ 数量】个风险点，其中【◆ 数量】个风险点在建设期真实发生并有预案应对。系统上线后【◆ 核心质量指标】达到预期。

不足之处：

一是评估主要依靠会议评审与经验判断，尚未引入量化的属性分析模型（如排队模型、可靠性框图）；

二是评估结论缺少工具化沉淀，决策记录以文档形式保存，检索与追溯不够便捷。

后续计划引入量化分析工具与架构决策记录（ADR）机制。

通过本项目我深刻体会到，架构评估的价值不在于证明架构正确，而在于\*\*在投入大量资源之前，把"可能的错误"找出来\*\*。

四、本项目填充点速查（术语对照表）

| 术语           | 本项目实例                                                                      |
|--------------|----------------------------------------------------------------------------|
| 性能敏感点        | DynamoDB 分区键设计； <code>fetchSize=200</code> ； <code>scan-limit=10000</code> |
| 可用性敏感点       | Session 存储位置；批量作业数据源隔离；Lambda 函数粒度                                         |
| 安全性敏感点       | 授权校验位置；凭证获取方式（STS vs 长期密钥）                                                 |
| 可修改性敏感点      | 契约定义位置；依赖版本声明方式；模块依赖方向                                                     |
| 可测试性敏感点      | 云依赖是否可本地模拟（LocalStack / TestContainers / moto）                             |
| 权衡点          | 混合存储；Serverless 同步；多语言契约生成                                                 |
| 风险点（已发生）     | 动态版本区间导致构建不可复现 → protobuf 启动失败                                             |
| 风险点（未发生）     | AWS SDK v1/v2 长期并存的维护成本累积                                                  |
| 非风险点         | Spring Session + Redis Cluster 会话外置                                        |
| 质量属性战术（可用性）  | 故障隔离（部署单元拆分、Lambda 按域拆分）                                                   |
| 质量属性战术（性能）   | 减少存储开销（读写分离）、协议优化（gRPC/HTTP2）                                              |
| 质量属性战术（可修改性） | 限制依赖（版本 force 锁定）、抽象（ <code>IGrpcService</code> 接口）                        |
| 质量属性战术（可测试性） | 本地云模拟（LocalStack）、接口分离（ <code>IGrpcService</code> ）                        |

五、可用难点素材（按适配度排序）

- 1. ② **protobuf 版本冲突** ★★ ★ —— 风险点 → 实际故障，论点完整
- 2. ⑤ **混合存储选型** ★★ ★ —— 最典型的权衡点
- 3. ⑥ **Serverless 同步** ★★ ★ —— 性能与一致性的冲突处理
- 4. ③ **AWS SDK v1/v2 并存** ★★ —— 可修改性的长期成本

六、踩坑提醒

| 坑              | 后果    | 正确做法                                      |
|----------------|-------|-------------------------------------------|
| 不提 ATAM 或提了不会用 | 明显不切题 | 背熟四阶段 + 四概念（敏感点/权衡点/风险点/非风险点）             |
| 术语用错           | 反而扣分  | 敏感点=关键性质；权衡点=影响多属性的敏感点；风险点=可能致问题的决策       |
| 只写「我们评估了」不写过程  | 空洞    | 按「场景收集 → 视图描述 → 属性分析 → 折中决策」四步写           |
| 不写量化目标         | 关键失分  | 质量属性场景必须有 <b>响应度量</b>                     |
| 说「所有质量属性都达到最优」 | 逻辑错误  | 质量属性天然相互制约，必须写权衡                          |
| 编造评估工具         | 高风险   | 本项目未见 Archimate 等工具使用证据；写「效用树 + 会议评审」是可行的 |
| 忘记写评估的产出       | 结构缺失  | 评估必须落到「识别了哪些点 + 形成了什么决策 + 有何效果」           |
| 与项目脱节，纯讲理论     | 判定为背书 | 每个理论点都要挂一个本项目实例                           |

💡 **本主题的备考捷径：**把上面「四、术语对照表」背下来。这张表把教材术语与你的项目一一对应，考场上只要题目沾「评估/质量属性/权衡」，就可以直接展开。这是本模板集中**理论性最强也最容易拿分**的一篇。

配套：[01- 微服务架构设计.md](#) · [03- 数据架构与混合存储.md](#) · [09- 架构演化与遗留系统改造.md](#)

论文模板 08 · 批处理架构设计

配套素材见 [00-项目素材库.md](#)。本主题**出现频率中等**，但因多数考生缺乏真实批处理经验，竞争压力小，是稳妥的备选题。

一、真题常见问法

变体 A（批处理整体）

请论述你参与的某系统中批处理（离线处理）部分的设计，包括作业调度、容错重跑、性能优化等方面。

变体 B（数据导入导出）

请论述你如何设计系统的数据导入/导出功能，如何保证大批量数据处理的正确性与效率。

变体 C（作业可靠性）

请论述你如何保障定时作业的可靠性，如何处理作业失败与幂等性问题。

**判断：**BASE HOME 在三个变体上都有真实素材—— 10 个 Spring Batch Job、双数据源读写分离、幂等控制、专用重试服务、Prometheus 监控、CSV 生成与 S3 上传。

## 二、摘要模板（约 300 字）

本文论述了本人参与的 H.I.S. BASE HOME 旅游业务平台中批处理架构的设计与实现。

该平台的批处理承担成本文件生成、自动邮件生成与发送、外部预约数据导入、主数据联动、ToDo 状态更新等核心离线业务，共 10 个 Spring Batch 作业。

本人担任架构师，负责批处理架构设计、作业可靠性保障与性能优化。

针对【批量作业与在线业务争抢数据库资源】、【消息推送重复发送风险】、【作业失败后的可恢复性】三大挑战，

本文从【读写分离双数据源】、【幂等与专用重试】、【参数化可重跑设计】三个维度进行设计：

为批处理配置独立的读写数据源并按包路径隔离 Mapper，

以幂等时间窗与独立重试服务杜绝重复推送，

将基准日期等关键参数显式参数化以保证结果可复现。

系统上线后【 填效果】，验证了方案的可行性。

## 三、正文五段框架

### 第 1 段·项目背景与批处理业务（约 400 字）

批处理论文开头要列清「有哪些作业」，体现真实经历。

BASE HOME 的批处理承载日终与定时的离线业务， 基于 Spring Batch 3.5.7 实现， 共 10 个作业：

| 作业                           | 业务含义             |
|------------------------------|------------------|
| ActivityCostFileCreateJob    | 活动原价成本文件生成       |
| AirCostFileCreateJob         | 机票成本文件生成         |
| HotelCostFileCreateJob       | 酒店成本文件生成         |
| AutomaticEmailCreateJob      | 自动邮件生成           |
| MailSendJob                  | 邮件发送             |
| ExternalReservationImportJob | 外部预约数据导入         |
| ImportMdmJob                 | 集团 MDM 主数据联动     |
| GenericQueryExecutionJob     | 通用查询执行（元数据驱动）    |
| TodoStatusUpdateJob          | MyPage ToDo 状态更新 |
| NoopJob                      | 测试用空作业           |

这些作业的业务特征差异很大：成本文件生成是**计算密集型**（需汇总大量费用明细）， 邮件发送是**IO 密集型**（依赖外部网关）， 数据导入则是**混合型**。批处理模块设置为 `web-application-type: none`， 不占用 Web 端口， 与在线业务在进程级完全隔离。

第 2 段 • 架构设计 (约 800 字, 本文核心)

第一层：资源隔离——读写分离双数据源

批处理最大的架构风险不是作业本身失败，而是**批量作业拖垮在线业务**。成本文件生成需要扫描大量费用明细，外部预约导入需要回写大量记录， 若与在线接口共享数据库连接池，夜间批次高峰会直接推高在线接口延迟。

我的方案是配置**双数据源**： - `ReaderDataSourceConfig` ( `spring.datasource.reader` ) —— 只读账号，承担大量查询 - `WriterDataSourceConfig` ( `spring.datasource.writer` , 标 `@Primary` ) —— 写账号，承担结果回写

通过**包路径隔离 Mapper** ( `repository/reader/` 与 `repository/writer/` )， 使读写走不同数据源，物理层面互不干扰。配合的优化还包括： - 事务隔离级别设为 `TRANSACTION_READ_COMMITTED`， 避免长事务期间持有大量间隙锁影响在线写入 - `mybatis.defaultFetchSize = 200` 控制批量游标，防止一次性载入海量结果集导致 JVM 内存溢出—— 这是批处理最典型的故障模式

**权衡说明**：双数据源增加了配置复杂度， 且跨数据源无法使用单一事务（需业务上避免跨源事务）， 但换来的在线业务稳定性收益远大于此成本。

第二层：作业组织与复用

10 个作业统一继承 `job/base/ManagedBaseJob` 基类，把作业的公共能力（参数解析、日志、异常处理）收敛到基类，各作业只关注自身业务逻辑。业务逻辑下沉到 `service/` 层的对应服务（`MailSendService`、`ImportMdmService`、`FileCreateService` 等），使作业类只承担编排职责，业务逻辑可被测试与复用。作业参数通过枚举统一定义（`JobParameterKey`）：`acquisitionRange`（取得范围）、`executionBaseDate`（执行基准日）、`todoDistinction`、`emailType`、`importTargetType`。

### 第三层：作业的可靠性设计

- **幂等控制**：`app.batch.idempotency-duration-minutes = 10` 提供时间窗去重，防止重复执行造成重复推送
- **专用重试服务**：`MailSendRetryService` 独立承担邮件重试，与业务解耦，重试策略可独立调整
- **事务粒度控制**：`service/transaction/*TransactionalService` 把事务边界限制在必要范围，避免大事务回滚造成连带重放
- **可观测性**：通过 **Prometheus PushGateway** 上报作业指标（`shutdown-operation: post` 确保短生命周期进程在退出前完成指标推送），使定时作业的运行状态可被监控系统采集与告警

### 第四层：元数据驱动的通用查询

`GenericQueryExecutionJob` 配合 `MGenericQuery*` 系列表，把查询逻辑外置为元数据配置，新增一类查询只需配置数据而无需改代码。这一设计显著提升了批处理的**可修改性**——业务侧新增报表类查询不再需要开发排期。

## 第 3 段 · 关键技术实现（约 700 字）

### 要素一：CSV 文件生成与上传

三类成本文件（ACT/AIR/HTL）使用 **SuperCSV 注解库** 实现，通过注解将实体字段直接映射为 CSV 列，导出列定义集中在实体上（`ActivityCostDownloadCsv`、`AirCostDownloadCsv`、`HotelCostDownloadCsv`），配合 `CsvDownloader` 接口与 `FileUploadService` 落 S3。**设计价值**：新增导出列时只需修改实体注解，不会波及导出逻辑代码——这是「限制变更影响范围」的可修改性战术。

### 要素二：外部数据导入

外部预约数据导入（`ExternalReservationImportJob`）通过 `RecvImportType` 枚举区分导入类型，经 `ExternalReservationImportService` 处理入库。集团主数据联动（`ImportMdmJob`）通过 `api/mdm/` 封装的认证与查询参数访问 MDM API，本地落为 26 张 `M*` 主数据表。

### 要素三：消息通知联动

批处理完成后需推送通知，通过 `SnsService` 发布到 SNS 主题（`app.services.aws.sns`），由下游 Lambda 消费。这种「批处理产出 → SNS → Lambda 消费」的模式，把批处理与通知渠道解耦：新增通知渠道无需修改批处理代码。

要素四：作业参数的显式化

我把 `executionBaseDate`（执行基准日）、`acquisitionRange`（取得范围）等关键参数定义在 `JobParameterKey` 枚举中并显式传入，而非在代码中直接取系统当前时间。这一设计带来两个收益：① **可重跑**——补数时指定历史日期即可重新生成，结果与当日运行一致；② **可测试**——测试可固定参数，使结果可预期。这对跨国系统尤其重要：服务日本本部（源库 JST）而容器运行于 UTC，若依赖系统时钟取「今天」，跨时区下极易取到错误的日期边界。

第 4 段 · 难点与解决（约 700 字）

批处理主题最佳组合：

| 优先级 | 难点               | 适配理由                          |
|-----|------------------|-------------------------------|
| 首选  | ⑦ 批量推送的幂等与重试     | 主题核心                          |
| 次选  | ④ MySQL 读写分离双数据源 | 直接对应「资源隔离」                    |
| 三选  | ⑧ 跨国多时区一致性       | <b>本主题的杀手级素材</b> ，直接对应「作业正确性」 |
| 加分  | ⑩ MDM 主数据同步      | 对应「数据导入」                      |

难点示例（⑧ 多时区，批处理视角）：

**问题：**系统服务日本本部，源数据库运行于 JST，而批处理容器环境为 UTC。这个差异在「取当天数据生成成本文件」这类作业上会直接造成错误——容器认为的「今天」在 JST 下可能还是「昨天」，反之亦然。后果是成本文件漏当天的数据或重复包含前一天的数据，而这类错误不会抛出异常，只会静默地产生错误结果。

**方案：**我的处理分两处。第一处是**存储口径统一**：数据库中的时间统一以 UTC 存储，展示与业务判断时按 JST 转换，转换动作集中到 `FormatUtils` 工具类，禁止业务代码中散落直接取当前时间的调用。第二处是**批处理参数化**：把 `executionBaseDate` 作为 `JobParameter` 显式传入而不依赖系统时钟，使作业的日期边界由调用方决定。这带来的额外收益是**可重放性**——指定同一基准日重跑，结果与当日运行完全一致，补数场景可靠。

**效果：**◆ 时区相关缺陷数量下降情况；作业结果可复现。

第 5 段 • 总结与不足 (约 200 字)

目前 10 个批处理作业已稳定运行【◆ 时长】，成本文件与邮件在【◆ 时限】内稳定完成，未发生重复推送事故。

不足之处：

一是幂等控制基于时间窗，理论上在窗口边界仍有极小概率的重复；

二是作业间的依赖关系（如邮件生成必须先于邮件发送）目前依靠调度时序保障，尚未引入显式的依赖编排。

后续计划引入基于持久化状态位的幂等机制与作业依赖编排。

通过本项目我认识到，批处理架构的关键不在于"批"的效率，而在于"失败之后能不能干净地重来"——幂等与可重跑，比处理速度更重要。

四、本项目填充点速查

| 论文要素      | 直接可写的内容                                                                                                 |
|-----------|---------------------------------------------------------------------------------------------------------|
| 框架        | Spring Batch 3.5.7 (Spring Boot 3.5.7)                                                                  |
| 作业数量      | 10 个 (含 1 个测试用 NoopJob )                                                                                |
| 作业基类      | job/base/ManagedBaseJob                                                                                 |
| 成本文件作业    | ActivityCostFileCreateJob / AirCostFileCreateJob / HotelCostFileCreateJob                               |
| 邮件作业      | AutomaticEmailCreateJob (生成) / MailSendJob (发送)                                                         |
| 导入作业      | ExternalReservationImportJob / ImportMdmJob                                                             |
| 通用查询      | GenericQueryExecutionJob + MGenericQuery* 元数据表                                                          |
| ToDo 作业   | TodoStatusUpdateJob                                                                                     |
| 读写分离      | ReaderDataSourceConfig / WriterDataSourceConfig ( @Primary )                                            |
| Mapper 隔离 | repository/reader/ 与 repository/writer/                                                                 |
| 事务隔离      | TRANSACTION_READ_COMMITTED (Hikari 配置)                                                                  |
| 批量游标      | mybatis.defaultFetchSize = 200                                                                          |
| 幂等        | app.batch.idempotency-duration-minutes = 10                                                             |
| 重试服务      | service/retry/MailSendRetryService                                                                      |
| 事务服务      | service/transaction/*TransactionalService                                                               |
| 作业参数      | JobParameterKey : acquisitionRange / executionBaseDate / todoDistinction / emailType / importTargetType |

| 论文要素   | 直接可写的内容                                                                                                             |
|--------|---------------------------------------------------------------------------------------------------------------------|
| 命令类型   | <code>CommandType</code> : DELETE / INSERT / UPDATE / DDL / NOT_FOUND                                               |
| CSV    | SuperCSV 注解库; <code>CsvDownloader</code> 接口; <code>FileUploadService</code> 上传 S3                                   |
| 输出类型   | <code>CostFile</code> (ACT/AIR/HTL) 、 <code>EmailType</code> 、 <code>SenderName</code> 、 <code>SubjectHeader</code> |
| 消息推送   | <code>SnsService</code> → SNS 主题; <code>MailSendService</code>                                                      |
| 监控     | Prometheus PushGateway ( <code>shutdown-operation: post</code> )                                                    |
| 运行模式   | <code>spring.main.web-application-type: none</code> (无 Web 容器)                                                      |
| 邮件发送配置 | <code>sender = log</code> (测试用日志代替实际发送) 、 <code>sleep-after-send = 0</code>                                         |
| 主数据联动  | <code>api/mdm/</code> (认证 + 查询参数) , <code>MdmContent</code> 枚举                                                      |
| 测试     | TestContainers + MySQL 镜像; <code>NoopJob</code> / <code>NoopService</code> 便于集成测试                                   |

## 五、可用难点素材（按适配度排序）

- 1. ⑦ 批量推送的幂等与重试 ★★ ★ —— 本主题首选
- 2. ⑧ 跨国多时区一致性 ★★ ★ —— 本主题的差异化杀手素材
- 3. ④ MySQL 读写分离双数据源 ★★ ★ —— 资源隔离
- 4. ⑩ MDM 主数据统一 ★★ —— 数据导入
- 5. ② protobuf 版本冲突 ★★ —— 依赖治理（可放"构建可靠性"）

## 六、踩坑提醒

| 坑                   | 后果   | 正确做法                              |
|---------------------|------|-----------------------------------|
| 只讲「用了 Spring Batch」 | 无深度  | 写清作业划分、参数化、幂等、重试、监控               |
| 不提幂等                | 关键失分 | 批处理论文 <b>必写幂等</b> ，这是可靠性核心        |
| 不提作业失败处理            | 漏考点  | 写「失败可观测 (Prometheus) + 可重跑 (参数化)」 |
| 只说「性能优化用了多线程」       | 高风险  | 本项目未见分区 (Partitioning) 并行证据，不要编造  |
| 编造 Quartz/XXL-Job   | 高风险  | 本项目未见独立调度框架，不要写；调度方式如实描述即可        |
| 忽略与在线业务的隔离          | 漏考点  | 读写分离是本项目真实亮点，务必写                  |
| 忘记写业务含义             | 显得空洞 | 每个作业都要说明它支撑什么业务（成本文件、邮件、ToDo...）  |

⚠ **技术真实性提醒：** 本项目**未见** Spring Batch 的分区（Partitioning）或多线程 Step 的证据，也**未见** XXL-Job / Quartz 等外部调度框架。论文中不要编造「我们使用了分片并行处理」。真实的性能手段是：**读写分离、游标控制、事务隔离级别、元数据驱动查询**——这四点已经足够撑起「性能优化」这一段。

配套：[03-数据架构与混合存储.md](#) · [04-高并发与高可用架构.md](#) · [02-云原生与FaaS架构.md](#)

## 论文模板 09 · 架构演化与遗留系统改造

配套素材见 [00-项目素材库.md](#)。本主题竞争压力小、容易出彩，因为大多数考生没有真实的架构演进经历。BASE HOME 的「AWS SDK v1/v2 渐进共存」「契约驱动三语言」都是极佳素材。

### 一、真题常见问法

#### 变体 A（架构演化）

请论述你参与的某系统架构的演化过程，包括演化的驱动力、演化策略以及演化过程中如何控制风险。

#### 变体 B（遗留系统改造）

请论述你如何对遗留系统进行改造，如何在不中断业务的前提下完成架构升级。

#### 变体 C（架构风格）

请论述你参与的某系统采用的架构风格，说明选择依据及该风格的优缺点。

#### 变体 D（可维护性/复用）

请论述你如何通过架构设计提升系统的可维护性与可复用性。

**判断：**变体 A/B 用「SDK v1/v2 渐进共存 + 契约统一」；变体 C 用「分层 + 事件驱动 + 服务化」的风格组合；变体 D 用「契约单一来源 + 版本统一治理 + 模块化」。

## 二、摘要模板（约 300 字）

本文论述了本人参与的 H.I.S. BASE HOME 旅游业务平台的架构演化过程。

该平台由早期单体应用逐步演进为多模块、多部署单元的架构，

期间需同时支撑 Java、Go、PHP 三类调用方，

并兼容大量基于旧版 AWS SDK 的既有代码。

本人担任架构师，负责演化路径规划、技术债治理与风险控制。

针对【新旧技术栈长期并存】、【接口契约在多语言环境下漂移】、

【演化过程不能中断业务】三大挑战，

本文遵循【渐进式演进】、【契约先行】、【依赖显式化】三个原则，

通过能力边界划分实现 AWS SDK v1/v2 的有序共存，

通过 protobuf 契约单一来源统一三端接口定义，

通过统一版本管理插件把隐式依赖转为显式契约。

系统在演化期间保持连续可用，【 填效果】，验证了策略的有效性。

## 三、正文五段框架

### 第 1 段·项目背景与演化驱动力（约 400 字）

演化类论文的核心是「驱动力」——为什么必须变？

BASE HOME 的架构演化并非为了技术而技术，而是由三类真实驱动因素推动：

**驱动一：调用方技术栈扩张** 系统最初仅服务 Java 侧的 Web 应用，随着集团平台化推进，周边 Go 平台与既有 PHP 系统也需接入。若为每种语言各写一份接口定义，字段口径必然漂移。这迫使我们在接口层做出根本性改变。

**驱动二：数据形态复杂化** 旅游业务的文档、亲权同意书、消息线程等数据字段演进频繁，关系型模型的 DDL 变更成本越来越高，促使我们引入 NoSQL 与对象存储，形成混合存储架构。

**驱动三：云平台能力演进** AWS SDK v2 提供了 v1 所没有的 DynamoDB 增强客户端（`DynamoDbEnhancedClient`），而既有代码大量基于 v1。全量重写风险过高，必须寻找演进路径。

这三类驱动力共同决定：**架构必须演化，但不能推倒重来。**

### 第 2 段·演化策略与过程（约 800 字，本文核心）

**第一原则：渐进式演进，拒绝大爆炸式重写**

面对 AWS SDK v1 到 v2 的迁移，我们评估了两条路径：

| 方案                        | 优点          | 风险                     |
|---------------------------|-------------|------------------------|
| 一次性全量迁移到 v2               | 技术栈彻底统一     | 涉及大量既有代码，回归范围不可控；业务不能停 |
| <b>v1/v2 按能力边界共存</b> （采纳） | 风险可控，新能力不受阻 | 双版本长期共存，依赖体积与维护成本上升    |

我们采纳了共存方案，并确立清晰的能力边界划分规则： - **新增能力一律走 v2**——DynamoDB 增强访问、STS AssumeRole 临时凭证 - **既有能力暂留 v1**——已稳定运行的 S3/STS 调用不做无谓改动 - **客户端构建集中收口**——在 `DynamoDbConfig`、`AwsConfig` 中统一构建，业务代码不直接创建客户端

这条规则的价值在于：它把「迁移」从一个需要排期的项目，变成了一个**随新增功能自然发生的渐进过程**。每新增一个功能，技术债就减少一点，而不需要专门立项。

**这是我在本项目最重要的架构判断：技术债的偿还方式，不一定是集中偿还，也可以是"增量稀释"。**

第二原则：契约先行，用单一来源约束多方

面对多语言调用方的接口漂移问题，我在 `base-home-spec` 模块中 确立 `protobuf` 为**唯一契约源**，通过构建流水线（`build.sh`：`build-java` → `build-go` → `build-php` → `generate-all`）生成 Java、Go、PHP 三语言产物及 OpenAPI 文档。

**演化视角的价值：**契约先行不仅解决了当下的接口一致性问题，更重要的它是**可演化性的基础**—— 因为契约集中，接口的版本演进、废弃标记、兼容性检查 都只需要在一个地方进行。分散的接口定义无法演化，只能重写。

第三原则：依赖显式化，把隐式契约变成可控状态

项目初期各模块独立声明依赖版本，部分采用动态区间（`protobufVersion = '[3,4]'`、`grpcVersion = 'latest.release'`）。这带来一个隐蔽的风险：**构建产物不可复现**—— 同一份代码在不同时间、不同机器上可能解析到不同版本。

我引入了独立的 `base-home-versions` Gradle 插件作为统一版本源，各模块通过 `apply plugin: 'base-home.versions'` 接入，并使用 `resolutionStrategy { force ... }` 锁定关键组件。

这不是一次预防性的重构——**风险已经真实发生：** `proto-google-common-protos` 被解析到 2.64.0-rc1 后，gRPC 服务启动直接失败。这次故障印证了「隐式版本契约」是架构层面的缺陷，而非偶发问题。

第 3 段 · 演化过程中的具体改造 (约 700 字)

改造一：数据存储的演化

从单一关系库演进为混合存储：Aurora MySQL 承载强一致交易（79 张表）、DynamoDB 承载高弹性半结构化数据（5 张表）、S3 承载文件实体。

**演化策略：**新数据类型直接落在新存储，既有数据不做迁移。例如文档与亲权同意书这类新增业务，直接以 `document` / `parental` 表落 DynamoDB，而既有交易数据保持在 MySQL。这样避免了大规模数据迁移的风险，同时让架构逐步具备弹性能力。

**代价与应对：**跨存储的一致性通过事件驱动链路达成最终一致——DynamoDB Stream / SQS 触发的 14 个 Lambda 函数负责将数据同步回 MySQL。这条链路本身就是「演化」的产物：它使新旧存储可以并存而不失一致。

改造二：在线与离线处理的分离

批量作业从与在线业务共享数据库，演进为双数据源读写分离（`spring.datasource.reader` / `writer`，按包路径隔离 Mapper）。这一改造的直接动因是：批处理峰值会推高在线接口延迟。改造后读写的物理隔离使两者互不干扰。

改造三：服务能力的沉淀与复用

随着 RPC 数量增长（当前 17 个），跨业务域的重复逻辑被识别并下沉：文档、消息线程、亲权同意书、序列号、旅行信息 5 类能力沉淀到 `service/shared/`，文件能力归入 `service/filesystem/`。服务端也从「每个 RPC 一个类」重构为「单一入口 `BaseHomeService` + `execute()` 统一分发」——横切关注点（日志、异常转换、上下文校验）得以集中处理。

第 4 段 • 难点与解决 (约 700 字)

架构演化主题最佳组合：

| 优先级 | 难点                      | 适配理由            |
|-----|-------------------------|-----------------|
| 首选  | ③ AWS SDK v1/v2 并存      | 直接对应「遗留系统改造」    |
| 次选  | ① 一份 proto 契约驱动三语言      | 直接对应「架构复用与可演化性」 |
| 三选  | ② protobuf 版本冲突（风险真实发生） | 直接对应「演化风险控制」    |
| 加分  | ⑤ 混合存储的形成过程             | 直接对应「数据架构演化」    |

难点示例（③，三段式）：

**问题：**DynamoDB 增强客户端只在 AWS SDK v2 提供，而既有代码大量基于 v1。全量迁移涉及面广、回归风险不可控，且业务不能停机等待。

**方案：**我判断这里的核心矛盾不是「技术选型」，而是「**如何在不停机、不失控的前提下完成技术栈演进**」。因此我没有选择一次性重写，而是按能力边界划分：新增能力走 v2，既有能力暂留 v1，客户端构建统一收口在配置类中，业务代码不直接创建客户端。这样做的深层价值是——**收口点就是未来的迁移点**。当某天要彻底移除 v1 时，只需改动配置类，而不需要遍历所有业务代码。

**效果：**◆ 新功能开发不再受 SDK 版本阻塞；◆ 凭证获取统一收口后，凭证类故障下降。更重要的是，技术债从「需要专门立项偿还」变成了「随新增功能自然稀释」。

第 5 段 · 总结与不足 (约 200 字)

整个演化过程历时【◆ 时间】，期间系统保持连续可用，未发生因演进导致的业务中断。

不足之处有两点：

一是 AWS SDK v1 与 v2 的并存状态尚未终结，长期维护成本仍然存在；

二是跨语言契约的破坏性变更尚未实现自动化的兼容性检测，目前依赖人工评审。

后续计划完成 SDK 版本的最终收敛，并引入契约兼容性校验。

通过本项目，我对架构演化形成了三点认识：

- ① 演化必须由业务驱动力推动，而非技术偏好；
- ② 渐进式演进的成本看似更高，但风险可控，而大爆炸式重写的风险往往是不可承受的；
- ③ 把隐式契约显式化，是让架构「可演化」的前提。

四、本项目填充点速查

| 论文要素   | 直接可写的内容                                            |
|--------|----------------------------------------------------|
| 演化主线   | 单体 → 多模块多部署单元；单一关系库 → 混合存储；单语言 → 三语言               |
| 技术债 1  | AWS SDK v1 (S3/STS) 与 v2 (DynamoDB Enhanced) 并存    |
| 技术债 2  | 动态版本区间 → 统一版本源 + resolutionStrategy force          |
| 技术债 3  | 分散接口定义 → protobuf 契约单一来源                           |
| 渐进策略   | 新增走新版、既有留旧版、客户端构建集中收口                              |
| 数据演进   | 新增业务直接落 DynamoDB/S3，不做大规模数据迁移                      |
| 一致性保障  | DynamoDB Stream / SQS → 14 个 Lambda → MySQL (最终一致) |
| 在线离线分离 | 双数据源 reader/writer + 包路径隔离 Mapper                  |

| 论文要素   | 直接可写的内容                                                                                         |
|--------|-------------------------------------------------------------------------------------------------|
| 服务重构   | 单入口 <code>BaseHomeService</code> + <code>execute()</code> 分发; <code>service/shared/</code> 沉淀复用 |
| 契约演进   | 约 90 个 proto 分 request/response/enums/type 四层, <code>type/</code> 约 42 个可复用消息                   |
| 版本治理   | <code>base-home-versions</code> Gradle 插件 ( <code>base-home.versions.gradle</code> )            |
| 版本对齐   | <code>his-common-spec</code> 2.6.0 在 Java/Go/PHP 三侧对齐                                           |
| 构建产物   | Java → Maven; Go → module zip; PHP → Composer; 文档 → MkDocs                                      |
| 环境治理   | DynamoDB 表名带环境前缀, 「1 账户 1 集群」                                                                   |
| 模块依赖方向 | web/service → common + spec; batch → common + mdm-spec                                          |
| 本地研发演进 | TestContainers (MySQL) + LocalStack ( <code>:4566</code> ) + moto (Python)                      |
| 测试支撑   | <code>NoopJob</code> / <code>NoopService</code> 便于集成测试与逐步替换                                     |

💡 「NoopJob / NoopService」是个隐藏的演化论据：项目中存在专门用于测试的空实现作业与服务，这说明架构在设计时就考虑了可替换性——写「可演化性」「可测试性」时可以用作佐证。

## 五、可用难点素材（按适配度排序）

- 1. ③ AWS SDK v1/v2 并存 ★★★★★ —— 本主题首选
- 2. ① 一份 proto 契约驱动三语言 ★★★★★ —— 可演化性与复用
- 3. ② protobuf 版本冲突（风险已发生） ★★★★★ —— 演化风险控制
- 4. ⑤ 混合存储的形成过程 ★★★★★ —— 数据架构演化
- 5. ④ 读写分离改造 ★★ —— 在线/离线分离演化

## 六、踩坑提醒

| 坑                  | 后果     | 正确做法                           |
|--------------------|--------|--------------------------------|
| 编造「从单体拆成微服务」的戏剧性故事 | 高风险    | 本项目是模块化多仓库结构，如实写「多模块 + 独立部署单元」 |
| 只讲技术不讲驱动力          | 空洞     | 演化必须有业务/技术驱动力支撑                |
| 只讲结果不讲过程           | 漏考点    | 必须写路径选择与权衡（为什么不大爆炸式重写）         |
| 不提遗留系统的风险          | 漏考点    | 遗留改造论文必须写「风险控制」——业务不中断怎么保证     |
| 编造容器化改造 / K8s 迁移   | 高风险    | 本项目未见 K8s 证据                   |
| 忘记写演化代价            | 缺乏架构思维 | 必须写并存状态的维护成本、双版本依赖体积等代价        |

| 坑        | 后果  | 正确做法                            |
|----------|-----|---------------------------------|
| 编造数据迁移规模 | 高风险 | 本项目是「新增业务落新存储」，不是「海量数据迁移」，如实写更稳 |

💡 **本主题的最大优势：**「渐进式演进」这个论点本身，就是软考官方教材强调的核心理念。你有一个**真实发生过的**版本冲突故障（难点 ②）来佐证「隐式契约是架构缺陷」这一判断，这种「预判风险 → 风险发生 → 已备预案」的完整叙事，在论文中非常有说服力，绝大多数考生写不出来。

配套：[01- 微服务架构设计.md](#) · [06- 系统集成与gRPC.md](#) · [07- 架构评估与质量属性.md](#)